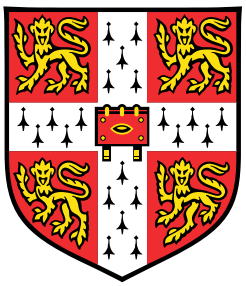
# Harnessing Transformations in Quantum Computing

## Fermion-to-Qubit Mappings, Quantum Metropolis Sampling and Trapped-Ion Experiments



**Oliver O'Brien**

Department of Applied Mathematics and Theoretical Physics
University of Cambridge

This dissertation is submitted for the degree of
*Doctor of Philosophy*

King's College

November 2026

I would like to dedicate this thesis to my loving parents ...

# Declaration

I hereby declare that except where specific reference is made to the work of others, the contents of this dissertation are original and have not been submitted in whole or in part for consideration for any other degree or qualification in this, or any other university. This dissertation is my own work and contains nothing which is the outcome of work done in collaboration with others, except as specified in the text and Acknowledgements. This dissertation contains fewer than 65,000 words including appendices, bibliography, footnotes, tables and equations and has fewer than 150 figures.

Oliver O'Brien
November 2026

# Acknowledgements

And I would like to acknowledge ...

# Abstract

This is where you write your abstract ...

# Table of contents

**List of figures** **xiii**

**List of tables** **xvii**

**Nomenclature** **xix**

**1 Introduction** **1**

1.0.1 Hybrid Mappings . . . 1

1.1 Preliminaries . . . 2

1.1.1 Jordan-Wigner Mapping . . . 2

1.1.2 Bravyi-Kitaev Mapping . . . 3

1.1.3 Auxiliary Qubit Mapping . . . 4

**2 Fermion-to-Qubit Mappings** **7**

2.1 Hybrid Mappings . . . 7

2.1.1 Hybrid mapping algorithm . . . 7

2.1.2 Algorithm analysis . . . 13

2.1.3 Discussion . . . 20

2.2 Local Jordan-Wigner transformations and topological sectors on the torus . . . 22

**3 Quantum Metropolis Sampling** **33**

3.1 Introduction . . . 33

3.2 Piecewise QSVT . . . 34

3.2.1 Piecewise exact linear block encoding . . . 35

3.2.2 Piecewise QSVT circuit . . . 38

3.2.3 State preparation . . . 39

3.2.4 Resource costs . . . 40

3.3 Applications . . . 41

3.3.1 Approximating nearly-analytic functions . . . 41

3.3.2 B-spline window . . . 43

3.3.3 Generic optimal algorithm . . . 45

3.4 Further improvements . . . 48
3.4.1 Block Encoding . . . 48
3.4.2 Priors . . . 49
3.5 Conclusion . . . 49
3.6 B-Spline Boosted Quantum Phase Estimation . . . 50

**References** **53**

**Appendix A Fermion-to-Qubit Mappings** **61**
A.1 Hybrid Mappings . . . 61
A.1.1 Steiner Tree Solver . . . 61
A.1.2 Visual guide to Hybrid mappings . . . 63

**Appendix B Installing the CUED class file** **67**

# List of figures


1.1 **Top:** Update set (green) and Flip set (blue) for $i = 3\ (011)$ on Fenwick tree with $N = 8$. **Bottom:** Parity set (red) for $i = 5\ (101)$ on Fenwick tree with $N = 8$. . . . . 4

2.1 Hybrid mapping with $2 \times 2$ cells showing the different sets of qubits involved in a fermionic operation between mode 6 and 27. Orange = targets, Blue = Flip set, Red = Parity set, Green = Update set . . . . . 10

2.2 Hybrid mapping with $2 \times 2$ cells showing the qubit operator representation of $\frac{1}{2}(a_{27} + a_{27}^\dagger)$. Orange = target, Blue = Flip set, Red = Parity set . . . . . 10

2.3 Hybrid mapping with $2 \times 2$ cells showing the qubit operator representation of $\frac{i}{2}(a_{27} - a_{27}^\dagger)$. Orange = target, Blue = Flip set, Red = Parity set . . . . . 10

2.4 Hybrid mapping with $2 \times 2$ cells showing the qubit operator representation of $\frac{1}{2}(a_6 + a_6^\dagger)$. Orange = target, Red = Parity set, Green = Update set . . . . . 10

2.5 Hybrid mapping with $2 \times 2$ cells showing the qubit operator representation of $\frac{i}{2}(a_6 - a_6^\dagger)$. Orange = target, Red = Parity set, Green = Update set . . . . . 11

2.6 Gates performed on edges of rows conditional upon the ancillas at the end of each row labeled with $C$. The dashed boxes represent the cells, whilst the circles inside them represent the roots of each cell, and the circles without boxes represent the ancillas. . . . 12

2.7 Examples from the two (for odd/even rows) series of gates applied conditional upon the ancillas labeled with $C$. Each element in the series is identical to the previous but shifted one row to the left (odd rows) or right (even rows). The first element in each series is the 2nd rightmost (odd rows) and 2nd leftmost (even rows). . . . . 12

2.8 Diagram showing how the Pauli-string representing a vertical hopping term (top) is modified by the Hybrid+ mapping algorithm. First, Z gates are added to all the ancillas with associated stabilisers that anti-commute with the string (middle). Then, a stabiliser shown in Figure A.1.3 is applied canceling out a large portion of the string to give the final local operation (bottom) . . . . . 14

2.9 The average Pauli-weight of fermionic operations on an $N \times N$ lattice against $N$ when using a given mapping. . . . . 17

2.10 The average range of each fermionic operation on an $N \times N$ lattice against $N$ when using a given mapping in the context of a square-lattice qubit connectivity. . . . . 17

2.11 The average Pauli-weight of a fermionic operation on an $32 \times 32$ lattice when using an $n \times n$ Hybrid/Hybrid+ mapping against $n$. The crosses denote the qubit-to-fermion ratio of the $n \times n$ Hybrid+ mapping for each choice of $n$. . . . . 18
2.12 The average range of each fermionic operation on an $32 \times 32$ lattice when using an $n \times n$ Hybrid/Hybrid+ mapping against $n$ in the context of a square-lattice qubit connectivity.The crosses denote the qubit-to-fermion ratio of the $n \times n$ Hybrid+ mapping for each choice of $n$. 18
2.13 The average range of each fermionic operation on an $64 \times 64$ lattice when using an $n \times n$ Hybrid/Hybrid+ mapping against $n$ in the context of a square-lattice qubit connectivity for $n = \{1,2,4,8,16,32,64\}$. These results include qubits solely acted on by SWAP gates. The crosses denote the qubit-to-fermion ratio of the $n \times n$ Hybrid+ mapping for each choice of $n$. . . . . 19
2.14 The average range of each fermionic operation on an $128 \times 128$ lattice when using an $n \times n$ Hybrid/Hybrid+ mapping against $n$ in the context of a square-lattice qubit connectivity for $n = \{1,2,4,8,16,32,64,128\}$. These results include qubits solely acted on by SWAP gates. The crosses denote the qubit-to-fermion ratio of the $n \times n$ Hybrid+ mapping for each choice of $n$. . . . . 19

3.1 Circuit implementing controlled version of piecewise exact linear block encoding where $L_{s_x} = 2^{l_{s_x}}$ and $l_{max} = \max_i(l_i)$. The slash circle control $\oslash$ is used to denote multiplexed control [72] (e.g. in this case it means that the action of the compute carry depends on the values of $k$ and $x$ in the corresponding registers). The mid circuit kets indicate the basis states per register with the full state being a summation over $a$ and $k$; after the application of compute carry these basis states are entangled. . . . . 36
3.2 Example of compute carry in Fig. 3.1 for $l_0 = 3$, $l_1 = 1$, $l_2 = 1$, $l_3 = 2$ and $l_4 = 4$. The unary iteration over these segments to copy the correct carry bit into the flag register is performed by controlled variable unary iteration [94]. Compared to Fig. 3.1, ancillas $\text{carry}_1$ to $\text{carry}_4$, and an ancilla unary iterator qubit are shown. The variable unary iteration contains further internal ancilla qubits. . . . . 37
3.3 Example of adder in Fig. 3.2 for $L_{max} = 16$, $N = 32$. It is important to note that the construction of the adder leaves the $x$ registers unaffected, so the variable unary iteration can operate directly on the $x$ register next. $x_4$ is not involved in this operation as $l_{max} = 4$ is less than $n = 5$ and it is not necessary to compute carries for $l > l_{max}$. The qubits have been ordered differently to Fig. 3.2 to make the ladder structure more apparent. . . . . 37
3.4 Circuit demonstrating how the variable unary iteration controlled rotations can be implemented by digitally loading the phase factors and adding into a phase gradient register. This technique avoids Clifford+T compilation of rotations[14], which would lead to a multiplicative factor $\log 1/\varepsilon$ in T complexity. . . . . 38

3.5 Piecewise indefinite parity QSVT circuit with 4 iterations to implement $S$ different polynomials of degree 2 on $S$ different segments of length $2^{l_{sx}}$. The multiplexed rotations are implemented efficiently using the variable unary iteration subroutine described in Section 3.2.1. . . . . 39
3.6 Numerical results demonstrating $O(\log\log\frac{1}{\delta})$ dependence of extra ancilla qubits for QPE when using both the best Kaiser Window and the best B-Spline window [2]. Comparison with the rectangular window demonstrates the improvement over “unboosted" QPE. Both the Kaiser Window and B-Spline window demonstrate an exponential improvement, though the Kaiser window has a slightly larger exponent. The green regions indicate target error rates of $\delta$ for which the same number of extra ancillas are required for both the Kaiser window and the B-Spline window. In these regions it is optimal to use the B-Spline window due to its lower preparation cost. This figure includes calculations for fractional qubit counts as it is possible to calculate $\delta$ as if we used a confidence interval not equal to a power of 2. . . . . 46
3.7 Numerical results demonstrating Toffoli cost of preparing the “best" $\beta$-Kaiser and $m$-th B-Spline windows for a given number of extra ancilla qubits. The parameters ($\beta$ and $m$) of the windows are chosen to minimise the probability of measuring the wrong energy when performing QPE with 25 base qubits. Kaiser window is costed using the state-of-the-art approach from Appendix E of [44] (derived from [79]), whereas the B-spline window is costed using our piecewise QSVT according the formulas given in 3.2.4 incorporating the improvements from 3.4.1. These results demonstrate up to 50x lower cost for the B-spline window. . . . . 47

A.1.1 Diagram illustrating how the *cell number* is assigned in a Z pattern to each cell in the Hybrid mapping. . . . . 63
A.1.2 Diagram illustrating how the *cell number* is assigned in a S pattern to each cell in the Hybrid+ mapping. The unnumbered circles represent the ancillas each connected to the cell directly below them. . . . . 63
A.1.3 Examples from the family of stabilisers created by applying the series of gates illustrated in 2.6 and 2.7. The family consists of a stabiliser for every ancilla which is a string of Z gates that follow the line of the S-pattern with either $XXY$ or $YXX$ gates surrounding the ancilla depending on whether the row is odd or even. . . . . 64
A.1.4 Diagrams showing how the portion of Pauli-strings acting on the roots of cells are modified and simplified by the Hybrid+ mapping algorithm . . . . . 65
A.1.5 Diagrams showing how the portion of a Pauli-string acting on the roots of the cells is modified and simplified by the Hybrid+ mapping algorithm . . . . . 66

# List of tables


2.1 Comparison of the performance (on an $N \times N$ square-lattice fermionic interaction graph) of select families of fermion-to-qubit mappings including state-of-the-art alongside our novel mapping $n \times n$ Hybrid and Hybrid+ mappings for $n \ll N$. $\mathscr{BK}_m$ and $\overline{\mathscr{BK}}_m$ represent the $O(\log m)$ maximum and average Pauli-weight of fermionic operators under the Bravyi-Kitaev mapping on $m$ fermionic modes. For each hybrid mapping, $n$ is fixed. For $n = 1$ the Hybrid mapping is equivalent to the Jordan-Wigner mapping, and the Hybrid+ mapping is equivalent to AQM. For $n = N$ both are equivalent to the Bravyi-Kitaev mapping. Intermediate values of $n$ combine the strengths of these mappings to give improved performance. The optimal choice of $n$ is dependant upon $N$ and must be found numerically (Figures 2.12,2.13). For $N = 64$, the optimal choice is $n = 8$. The $8 \times 8$ Hybrid mapping gives 98% shorter Pauli-strings than the Jordan-Wigner mapping, and the $8 \times 8$ Hybrid+ mapping only requires 1.016 qubits per fermion. This is 94% fewer ancillae than the super-compact mapping (which requires the fewest ancillae of all known local mappings). *These values are estimated using numerical evaluation. . . . . 8

2.2 Mapping of topological sectors of fermions to spins on the chain dependent on the choice of boundary condition (BC 1) for the duality MPO. These sectors can then be mapped again to the dual spins by the application of a Kramers-Wannier MPO with a suitable choice of boundary condition (BC 2). The composed mapping from fermions to dual spins is equivalent to the canonical Jordan-Wigner mapping with one single boundary condition (Unified BC). . . . . 27

2.3 Table illustrating how different boundary conditions of the PEPO on the torus map different charge sectors and produce different twists. The $\hat{Z}_{p_V}$ and $\hat{Z}_{p_H}$ twists are vertical and horizontal loops of $\hat{Z}$ operators on the physical fermions. The $Z_v X_h / Z_h X_v$ twist (with subscript $v/h$ indicating whether the operator is applied on a vertical or horizontal leg) constitutes a single unit cell of $X_H$ or $X_V$ depending on whether the twist was taken in the horizontal or vertical direction respectively. . . . . 29

3.1 Asymptotic Toffoli costs for loading these functions into the amplitudes of states on $n$ qubits with an error tolerance of $\varepsilon$. The tail probability $\delta$ when utilising the $m$-th B-spline window in QPE is given by $O(\exp(-m))$ as shown by Lemma 3. . . . . 45

# Nomenclature

**Roman Symbols**

| | |
|---|---|
| $F$ | complex function |

**Greek Symbols**

| | |
|---|---|
| $\gamma$ | a simply closed curve on a complex plane |
| $\imath$ | unit imaginary number $\sqrt{-1}$ |
| $\pi$ | $\simeq 3.14\ldots$ |

**Superscripts**

| | |
|---|---|
| $j$ | superscript index |

**Subscripts**

| | |
|---|---|
| 0 | subscript index |
| crit | Critical state |

**Other Symbols**

| | |
|---|---|
| $\oint_\gamma$ | integration around a curve $\gamma$ |

**Acronyms / Abbreviations**

| | |
|---|---|
| ALU | Arithmetic Logic Unit |
| BEM | Boundary Element Method |
| CD | Contact Dynamics |
| CFD | Computational Fluid Dynamics |
| *CIF* | Cauchy's Integral Formula |
| CK | Carman - Kozeny |

DEM Discrete Element Method

DKT Draft Kiss Tumble

DNS Direct Numerical Simulation

EFG Element-Free Galerkin

FEM Finite Element Method

FLOP Floating Point Operations

FPU Floating Point Unit

FVM Finite Volume Method

GPU Graphics Processing Unit

LBM Lattice Boltzmann Method

LES Large Eddy Simulation

MPM Material Point Method

MRT Multi-Relaxation Time

PCI Peripheral Component Interconnect

PFEM Particle Finite Element Method

PIC Particle-in-cell

PPC Particles per cell

RVE Representative Elemental Volume

SH Savage Hutter

SM Streaming Multiprocessors

USF Update Stress First

USL Update Stress Last

# Chapter 1

# Introduction

### 1.0.1 Hybrid Mappings

Simulating quantum systems is one of the most promising applications of quantum computing in the near future [36, 66, 89]. The class of fermionic systems is particularly important and simulating them has an impressive range of applications: from problems in quantum chemistry [77, 93], to the Hubbard model in condensed matter physics [2], to Lattice Gauge Theories [9, 21] and beyond. Classical methods applied to these problems do not scale favourably with the system size, whereas quantum techniques (e.g. VQE [80], Trotterization [108] and others [16, 38, 90]) open up efficient avenues for studying static and dynamic properties of fermionic systems. In recent years, there has been a considerable experimental effort to simulate small fermionic systems on a quantum computer [10, 63].

One of the crucial challenges facing these simulations is encoding the anti-commuting fermionic variables as operators acting on the qubit degrees of freedom. A desirable transformation would map interactions in the fermion picture to qubits resulting in (quasi-) local operators for the latter side. Furthermore, it should be engineered in a way to substantially reduce gate overhead in a circuit model – leading to faster simulation times and more robustness with respect to errors.

There is currently no known fermion-to-qubit mapping that preserves perfect locality and consumes no auxiliary resources (in the form of qubits or the extra gate overhead) for an arbitrary interaction graph. However, in the last decade there has been remarkable progress towards designing efficient mappings [111, 98, 49, 16, 96, 31, 25, 101] and exploring new ideas that would improve the existing ones [27]. A summary of select mappings (including the most efficient ones) can be found in Table 2.1. These mappings fall into two main categories: non-local mappings which require no additional resources but produce gate counts dependant on the total number of fermionic modes; local mappings which use additional resources (ancilla qubits) to produce constant gate counts independent of the number of fermionic modes. These local mappings can be physically interpreted as lattice gauge theories [24]. The best-known "explicitly constructed" local mapping is due to Chen and Xu [25]. It uses the fewest qubits per fermion mode (1.25) while producing Hamiltonians with $k$-local average and maximum Pauli string, for constant $k$. There do exist general constructions with lower ratios that only require $1+\frac{1}{k}$ qubits per fermion however they have the trade-off of a Pauli weight growing with $O(k)$.

In this work, we introduce a family of parametrized Hybrid mappings (Section 2.1.1) that combine the relaxed connectivity constraints of the Jordan-Wigner mapping and the increased locality of the Bravyi-Kitaev mapping to produce drastically reduced gate counts [107]. In the context of limited qubit connectivity (Section 2.1.2.3) the Hybrid mapping produces gate counts that scale with $\frac{N}{2n}$ compared to $\frac{N}{2}$ for the Jordan-Wigner and Bravyi-Kitaev mappings on an $N \times N$ lattice (where $n \ll N$). In the regime of all-to-all connectivity the Hybrid mapping gate count scales with $\frac{N}{2n^2}$ (Section 2.1.2.1) even further outperforming the unaffected Jordan-Wigner mapping. However, in the absence of connectivity constraints the Bravyi-Kitaev mapping gate count scales with $O(\log N)$. Despite this our numerical analysis in Section 2.1.2.4 shows the Hybrid mapping still results in lower gate counts than the Bravyi-Kitaev mapping for small lattices (e.g. lattices smaller than $164 \times 164$ for $n = 4$).

We also construct a Hybrid+ family of parametrized local mappings (Section 2.1.1.1) which only require $1 + \frac{1}{n^2}$ qubits per fermionic mode. Our construction interpolates between the Auxillary Qubit mapping [111, 101] and the Bravyi-Kitaev mapping by setting $n = 1$ for the former and $n = N$ for the latter. For all the intermediate $n$, our construction results in novel highly efficient mappings that produce a dramatic reduction in the number of auxiliary qubits required, while simultaneously generating local Pauli strings. We illustrate examples for $n = 4$ and $n = 8$ which demonstrate 1.0625 and 1.0156 qubit-to-fermion ratios compared to the best existing local mapping [25] which achieves a 1.25 qubit-to-fermion ratio.

The key insight of our work is that it is possible to apply seemly incompatible mappings on different scales such that their relative strengths complement each other. We combine the efficiency of the Bravyi-Kitaev mapping with the low connectivity restraints of the Jordan-Wigner mapping to produce a far superior Hybrid mapping. On a small scale (within each cell) we use the Bravyi-Kitaev mapping (in few fermion regime when it's connectivity restraints are not too large), but when we zoom out to a larger scale we use the Jordan-Wigner mapping between the cells. By varying the scale at which each mapping is dominant we are able to find an optimal mapping for a given quantum computer.

## 1.1 Preliminaries

In this section, we introduce the mappings our Hybrid schemes extend: the Jordan-Wigner Mapping (Section 1.1.1), the Bravyi-Kitaev Mapping (Section 1.1.2), and the Auxiliary Qubit Mapping (Section 1.1.3).

### 1.1.1 Jordan-Wigner Mapping

The Jordan-Wigner transform [61] is arguably the oldest known fermion-to-qubit mapping. It stores the occupation number of the $i$-th fermionic mode in the $i$-th qubit. Therefore, all the non-local behaviour (the

parity information) has to be encoded in the operators [27]:

$$a_i \to \left(\bigotimes_{k=1}^{i-1} Z_k\right) \sigma_i^-$$
$$a_i^\dagger \to \left(\bigotimes_{k=1}^{i-1} Z_k\right) \sigma_i^+$$

where

$$\sigma_i^- = \frac{1}{2}(X_i + iY_i) = |0\rangle_i \langle 1|_i$$
$$\sigma_i^+ = \frac{1}{2}(X_i - iY_i) = |1\rangle_i \langle 0|_i$$

The $\bigotimes_{k=1}^{i-1} Z_k$ term is necessary to apply the parity phase shift $(-1)^{\sum_{k=0}^{i-1} f_k}$, where $f_k$ are defined below. These operators act in exactly the same way on a qubit spin basis as the fermionic creation and annihilation operators act upon the Fock basis in Eqns (1.1) and (1.2).

$$a_p |f_0, \cdots, f_i, \cdots, f_{M-1}\rangle = \delta_{f_p,1}(-1)^{\sum_{i=0}^{p-1} f_i} |f_0, \cdots, f_p \oplus 1, \cdots, f_{M-1}\rangle \tag{1.1}$$

$$a_p^\dagger |f_0, \cdots, f_i, \cdots, f_{M-1}\rangle = \delta_{f_p,0}(-1)^{\sum_{i=0}^{p-1} f_i} |f_0, \cdots, f_p \oplus 1, \cdots, f_{M-1}\rangle \tag{1.2}$$

### 1.1.2 Bravyi-Kitaev Mapping

Our construction may be regarded as a particular extension of the Bravyi-Kitaev mapping [17]. Previous extensions of the Bravyi-Kitaev mapping have combined the Bravyi-Kitaev fast and super-fast mappings to logarithmically reduce the Pauli weight of fermionic operators [99].

The Bravyi-Kitaev mapping stores the parity of fermionic modes in a non-local Fenwick tree data structure [49]. A Fenwick tree [35] is a partial ordering of binary representations where every node is a copy of its parent with one of the 1s changed to a 0 as shown in Figure 1.1. In this section we outline how the Bravyi-Kitaev mapping encodes $m$ fermionic modes:

First, each fermionic mode is assigned a corresponding qubit and a node in a Fenwick tree. Every qubit stores the total parity of the fermionic modes below it in the Fenwick tree. This allows fermionic operators to be constructed with Pauli-weight $O(\log m)$.

These operators (as shown in [96]) are constructed using three easily calculated subsets of the Fenwick tree: the Update ($U$), Parity ($P$), and Flip ($F$) sets.

$$\begin{aligned} a_i &\to X_{U_i} \sigma_i^- Z_{P_i/F_i} \\ &\to \frac{1}{2}\left(\bigotimes_{j\in U_i} X_j \otimes X_i \otimes \bigotimes_{j\in P_i} Z_j + i \bigotimes_{j\in U_i} X_j \otimes Y_i \otimes \bigotimes_{j\in P_i/F_i} Z_j\right) \end{aligned}$$

$$a_i^\dagger \to X_{U_i}\sigma_i^+ Z_{P_i/F_i}$$
$$\to \frac{1}{2}\left(\bigotimes_{j\in U_i} X_j \otimes X_i \otimes \bigotimes_{j\in P_i} Z_j - i\bigotimes_{j\in U_i} X_j \otimes Y_i \otimes \bigotimes_{j\in P_i/F_i} Z_j\right)$$

Each of the above sets is defined (similarly to the original formulation [96]) as follows :

Use the binary representation: $\alpha = \alpha_{l-1}\dots\alpha_0$ and $\beta = \beta_{l-1}\dots\beta_0$, where index 0 denotes the most significant bit. Define $\prec$ to represent the partial ordering of the Fenwick tree. Fermionic modes are zero-indexed.

**Definition 1.** *Let the Update Set $U(\alpha)$ be such that $\beta \in U(\alpha)$ if and only if for some $i_0$, $\alpha_{i_0} = 0$ and $\alpha_i = \beta_i$ for $i \geq i_0$ and $\beta_i = 1$ for $i \leq i_0$. Equivalent to $\beta \in U(\alpha)$ if and only if $\alpha \prec \beta$.*

**Definition 2.** *Let the Parity set $P(\alpha)$ contain elements $\beta \in P(\alpha)$ if and only if for some $i_0$: $\alpha_{i_0} = 1$, $\beta_{i_0} = 0$, $\beta_i = \alpha_i$ for $i > i_0$ and $\beta_i = 1$ for $i < i_0$, then $\sum_{j\in P(i)} q_j = \sum_{j<i} f_j$*

**Definition 3.** *Let the Flip set $F(\alpha)$ contain elements $\beta \in F(\alpha)$ if and only if for some $i_0$: $\beta_{i_0} = 0$, $\beta_i = \alpha_i$ for $i \neq i_0$ and $\alpha_i = 1$ for $i \leq i_0$, then $\sum_{j\in F(i)} q_j = \sum_{j\prec i} f_j$*

Examples of these sets are given in Figure 1.1. We present three simple algorithms for generating these sets which will later be used in our construction: GENERATEUPDATESET, GENERATEPARITYSET, and GENERATEFLIPSET.

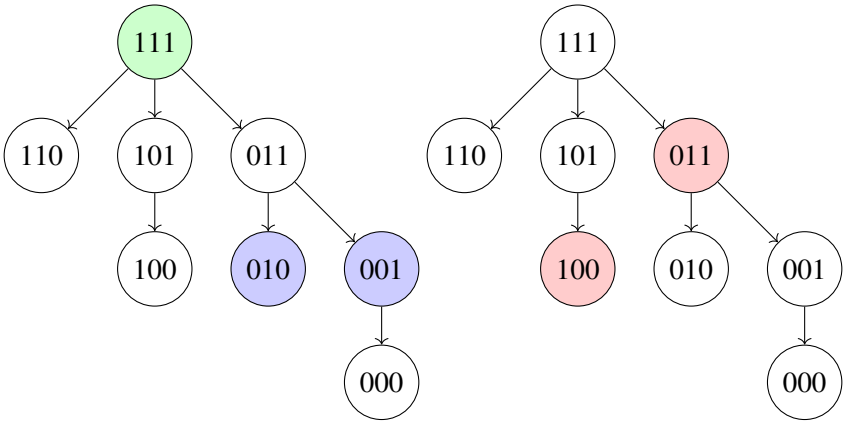



Fig. 1.1 **Top:** Update set (green) and Flip set (blue) for $i = 3$ $(011)$ on Fenwick tree with $N = 8$. **Bottom:** Parity set (red) for $i = 5$ $(101)$ on Fenwick tree with $N = 8$.

### 1.1.3 Auxiliary Qubit Mapping

Though originally pioneered by [111], here we recall the formalization of the auxiliary mappings from [101]. The principle behind auxiliary mapping schemes is to create stabilisers which are similar to non-local interaction terms in the Hamiltonian, so they can be used to cancel each other out. For

**Algorithm 1** GENERATEUPDATESET

**Input:** Fermionic enumeration: $\alpha$, Number of fermionic modes: $n$
**Output:** Update Set $U_\alpha$

updateSet = { }
$l \leftarrow$ length of binary representation of $\alpha$
$\beta \leftarrow \alpha$
**for** $i$ counting from 0 to $l-1$ **do**
    **if** $\alpha_i$ is 0 **then**
        $\beta \leftarrow \beta_{l-1} \ldots \beta_{i+1} 1 \beta_{i-1} \ldots \beta_0$
        **if** $\beta < n$ **then**
            Add $\beta$ to updateSet
        **end if**
    **end if**
**end for**

**Algorithm 2** GENERATEPARITYSET

**Input:** Fermionic enumeration: $\alpha$
**Output:** Parity Set $P_\alpha$

paritySet = { }
$l \leftarrow$ length of binary representation of $\alpha$
**for** $i$ counting down from $l-1$ to 0 **do**
    **if** $\alpha_i$ is 1 **then**
        $\beta \leftarrow \alpha_{l-1} \alpha_{l-2} \ldots \alpha_{i+1} 01 \ldots 1$
        Add $\beta$ to paritySet
    **end if**
**end for**

**Algorithm 3** GENERATEFLIPSET

**Input:** Fermionic enumeration: $\alpha$
**Output:** Flip Set $F_\alpha$

flipSet = { }
$l \leftarrow$ length of binary representation of $\alpha$
$i \leftarrow 0$
**while** $\alpha_i$ is 1 **do**
    $\beta \leftarrow \alpha_{l-1} \ldots \alpha_{i+1} 0 \alpha_{i-1} \ldots \alpha_0$
    Add $\beta$ to flipSet
    $i \leftarrow i+1$
**end while**

example, consider an $n$-qubit Hamiltonian consisting of $m$ Pauli-strings $h_i$ some of which contain a non-local Pauli-string $p$:

$$H = \sum_{i=0}^{m} h_i$$

- Let $|\psi\rangle_S$ be an arbitrary state on the $n$-qubit system ($\mathscr{H}_S$)
- Introduce an ancillary qubit $|0\rangle_A$ ($\mathscr{H}_A$)
- Entangle the ancilla with the $n$-qubit state by applying a given unitary $V$

$$V|\psi\rangle_S|0\rangle_A = |\tilde{\psi}\rangle_{SA}$$

- Find $\sigma$ such that $(p \otimes \sigma)$ is a stabiliser:

$$(p \otimes \sigma)|\tilde{\psi}\rangle_{SA} = |\tilde{\psi}\rangle_{SA}$$

- Modify $H$ so it commutes with the stabiliser (1.3) and thus acting with the modified Hamiltonian $\tilde{H}$ on a stabiliser state always produces another stabiliser state. This involves adding an operator $\kappa$ to each term $h_i$ in the Hamiltonian that anti-commutes with $(p \otimes \sigma)$ to produce a commuting term $h_i^{(\kappa)} = h_i \otimes \kappa$. This modification must be such that its action on the entangled system is equivalent to the action of the original Hamiltonian on the $n$-qubit state (1.4).

$$H^{(\kappa)}(p \otimes \sigma)|\tilde{\psi}\rangle_{SA} = (p \otimes \sigma)H^{(\kappa)}|\tilde{\psi}\rangle_{SA} \tag{1.3}$$

$$H^{(\kappa)}|\tilde{\psi}\rangle_{SA} = V(H \otimes I)|\psi\rangle_S|0\rangle_A \tag{1.4}$$

- Now we can use the stabiliser to cancel out the non-local $p$ string in the Hamiltonian to get a simplified Hamiltonian $\tilde{H}$ which is logically identical.

$$\tilde{h}_i = \begin{cases} h_i^{(\kappa)}(p \otimes \sigma) & p \in h_i \\ h_i^{(\kappa)} & p \notin h_i \end{cases}$$

$$\tilde{H} = \sum_{i=0}^{m} \tilde{h}_i$$

To perform this procedure to remove multiple non-local strings $p_i$, we can simply repeat the process with additional ancillas. However as the stabilisers $p_i \otimes \sigma_i$ must share common stabiliser states, it is only possible to simultaneously remove commuting strings.

# Chapter 2

# Fermion-to-Qubit Mappings

## 2.1 Hybrid Fermion-to-Qubit Mappings

We present two novel mapping algorithms: Hybrid mapping (Section 2.1.1) and Hybrid+ mapping (Section 2.1.1.1). The Hybrid mapping combines the low connectivity constraints of the Jordan-Wigner mapping with the increased locality of the Bravyi-Kitaev mapping. The Hybrid+ mapping builds on the strengths of the Hybrid mapping and expands the mapping to include aspects of AQM [111, 101] to increase the locality using ancilla qubits. In this section we outline their construction in detail, and then in Section 2.1.2 we will analyse their performance. Both these mappings have been designed for a square-lattice fermionic interaction graph. For the sake of clarity, they are both described with $n \times n$ cells, however, it is trivial to extend them to rectangular $l \times h$ cells. It is also possible to perform these mappings with irregular varying cell sizes. We note that it is possible to extend this mapping to any connectivity or fermionic interaction graph, one must simply make an appropriate choice of cells to split the graph into and then pick an order in which to connect them via Jordan-Wigner.

### 2.1.1 Hybrid mapping algorithm

**Input:** *$N \times N$ Fermion Square-Lattice Interaction Graph, Cell Shape $n \times n$, $1 \leq n \leq N$* and $n$ is a factor of $N$
**Output:** *Pauli-strings corresponding to interaction edges, Enumerated square-lattice of qubits*
**Summary:** This mapping effectively splits up the square lattice into a series of Fenwick trees (Bravyi-Kitaev maps) and connects them via a Jordan-Wigner mapping on the roots, as demonstrated in Figures 2.1 to 2.5. These diagrams show how creation and annihilation operators are mapped to Jordan-Wigner like $ZZ \ldots ZX$ strings on the roots of previous trees and Bravyi-Kitaev like strings within the current tree.
**Algorithm:**

1. Divide the square lattice of fermions into cells of size $n \times n$

| | Qubit to fermion ratio | Max Pauli string | Avg. Pauli string | Avg. Range | Max Range |
|---|---|---|---|---|---|
| Jordan-Wigner [61] | 1 | $2N$ | $\frac{N}{2}+\frac{3}{2}$ | $\frac{N}{2}+\frac{3}{2}$ | N+1 |
| Bravyi-Kitaev [16] | 1 | $\mathscr{BK}_{N^2}$ | $\overline{\mathscr{BK}}_{N^2}$ | $O(\frac{N}{2})$* | O(N) |
| $n\times n$ **Hybrid** | 1 | $\frac{2N}{n}+2\mathscr{BK}_{n^2}$ | $\frac{N}{2n^2}+\frac{n+1}{n}\overline{\mathscr{BK}}_{n^2}$ | $O(\frac{N}{2n}+\frac{n}{2})$ | $O(\frac{N}{n}+n)$ |
| Derby-Klassen [31] | 1.5 | 3 | 3 | 3 | 3 |
| Super-compact [25] | 1.25 | 6 | $O(1)$ | $O(1)$ | O(1) |
| AQM [111, 101] | $2-\frac{1}{N}$ | 6 | $O(1)$ | $O(1)$ | O(1) |
| $n\times n$ **Hybrid+** | $1+\frac{1}{n^2}$ | $3+2\mathscr{BK}_{n^2}$ | $\frac{2}{n}+\frac{1}{n^2}+\frac{n+1}{n}\overline{\mathscr{BK}}_{n^2}$ | $O(\frac{n}{2})$* | $O(n)$ |
| Low-ratio mapping [25, 101] | $1+\frac{1}{k}$ | $O(k)$ | $O(k)$ | $O(k)$ | $O(k)$ |

Table 2.1 Comparison of the performance (on an $N\times N$ square-lattice fermionic interaction graph) of select families of fermion-to-qubit mappings including state-of-the-art alongside our novel mapping $n\times n$ Hybrid and Hybrid+ mappings for $n\ll N$. $\mathscr{BK}_m$ and $\overline{\mathscr{BK}}_m$ represent the $O(\log m)$ maximum and average Pauli-weight of fermionic operators under the Bravyi-Kitaev mapping on $m$ fermionic modes. For each hybrid mapping, $n$ is fixed. For $n=1$ the Hybrid mapping is equivalent to the Jordan-Wigner mapping, and the Hybrid+ mapping is equivalent to AQM. For $n=N$ both are equivalent to the Bravyi-Kitaev mapping. Intermediate values of $n$ combine the strengths of these mappings to give improved performance. The optimal choice of $n$ is dependant upon $N$ and must be found numerically (Figures 2.12,2.13). For $N=64$, the optimal choice is $n=8$. The $8\times 8$ Hybrid mapping gives 98% shorter Pauli-strings than the Jordan-Wigner mapping, and the $8\times 8$ Hybrid+ mapping only requires 1.016 qubits per fermion. This is 94% fewer ancillae than the super-compact mapping (which requires the fewest ancillae of all known local mappings). *These values are estimated using numerical evaluation.

2. Assign a *cell number* $(c_i)$ to each of these cells in a Z pattern starting with the top-left most cell (illustrated by Figure A.1.1 in the Appendix). This enumeration reflects the Jordan-Wigner-esque properties of the mapping.

3. Within these cells assign each fermion a *mode number* $(f_i)$ from 1 to $n \times n$ in a Z pattern starting with the top-left most fermion. We can now refer to the $i$-th fermion in the graph via the coordinate $(c_i, f_i)$.

4. Repeat this procedure on a square qubit lattice of the same size so for every fermion there is an equivalent qubit labeled with the same coordinate

5. For each interaction edge in your fermion interaction graph between $(c_1, f_1)$ and $(c_2, f_2)$ generate the Pauli-string representation:

   (a) Generate the update $U_i$, flip $F_i$, and parity $P_i$ sets for $f_1$ and $f_2$ independently in a Fenwick tree of size $n \times n$
   $U_i = (c_i, \text{GENERATEUPDATESET}(f_i, n \times n))$
   $F_i = (c_i, \text{GENERATEFLIPSET}(f_i))$
   $P_i = (c_i, \text{GENERATEPARITYSET}(f_i))$

   (b) Add the roots of every cell lower in the Z pattern than $c_1$ and $c_2$ to their respective parity sets.
   FOR ALL $c_j < c_1$ ADD $(c_j, n \times n)$ TO $P_1$
   FOR ALL $c_j < c_2$ ADD $(c_j, n \times n)$ TO $P_2$

   (c) Pauli-string representation is given by:

$$
\begin{aligned}
a_1 a_2^\dagger &\to X_{U_1} \sigma_1^- Z_{P_1/F_1} X_{U_2} \sigma_2^+ Z_{P_2/F_2} \\
&= \frac{1}{4} \left( \bigotimes_{j \in U_1} X_j \otimes X_{(c_1,f_1)} \otimes \bigotimes_{j \in P_1} Z_j + ß \bigotimes_{j \in U_1} X_j \otimes Y_{(c_1,f_1)} \otimes \bigotimes_{j \in P_1/F_1} Z_j \right) \\
&\left( \bigotimes_{j \in U_2} X_j \otimes X_{(c_2,f_2)} \otimes \bigotimes_{j \in P_2} Z_j - i \bigotimes_{j \in U_2} X_j \otimes Y_{(c_2,f_2)} \otimes \bigotimes_{j \in P_2/F_2} Z_j \right) \\
&= \frac{1}{4} \bigotimes_{j \in (U_1 \oplus U_2)} X_j \bigotimes_{j \in (P_1/F_1 \oplus P_2/F_2)} Z_j \otimes \left( X_{(c_1,f_1)} \otimes \bigotimes_{j \in F_1} Z_j + i Y_{(c_1,f_1)} \right) \left( X_{(c_2,f_2)} \otimes \bigotimes_{j \in F_2} Z_j - i Y_{(c_2,f_2)} \right)
\end{aligned}
$$

It is important to note that while we have presented the Hybrid mapping in terms of the even fermionic operators (keeping inline with the Hybrid+ mapping which can only represent even fermionic operators), it can also admit a representation of the whole fermionic space. Figures 2.1 to 2.5 illustrate in detail how the odd fermionic operators are mapped, and it is clear how to construct the even fermionic operators by multiplying these together.

#### 2.1.1.1 Hybrid+ mapping algorithm

**Input:** *$N \times N$ Fermion Square-Lattice Interaction Graph, Cell Shape $n \times n$, $n \times n = 2^k$* for some integer $k$, $n$ is a factor of $N$

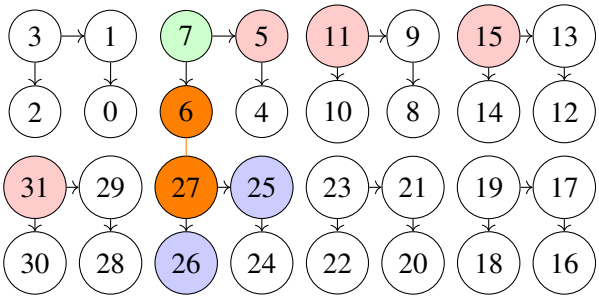



Fig. 2.1 Hybrid mapping with $2 \times 2$ cells showing the different sets of qubits involved in a fermionic operation between mode 6 and 27. Orange = targets, Blue = Flip set, Red = Parity set, Green = Update set

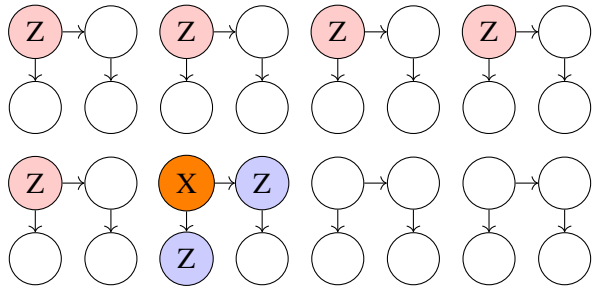



Fig. 2.2 Hybrid mapping with $2 \times 2$ cells showing the qubit operator representation of $\frac{1}{2}(a_{27} + a_{27}^{\dagger})$. Orange = target, Blue = Flip set, Red = Parity set

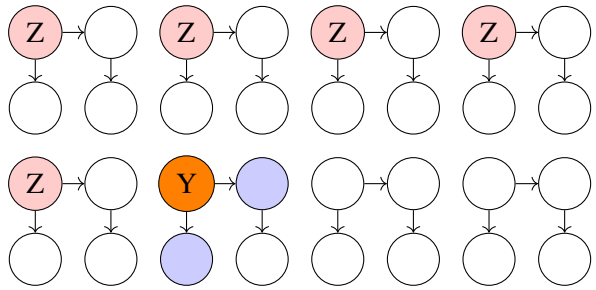



Fig. 2.3 Hybrid mapping with $2 \times 2$ cells showing the qubit operator representation of $\frac{i}{2}(a_{27} - a_{27}^{\dagger})$. Orange = target, Blue = Flip set, Red = Parity set

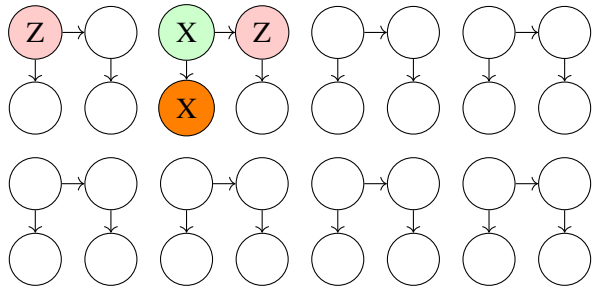



Fig. 2.4 Hybrid mapping with $2 \times 2$ cells showing the qubit operator representation of $\frac{1}{2}(a_6 + a_6^{\dagger})$. Orange = target, Red = Parity set, Green = Update set

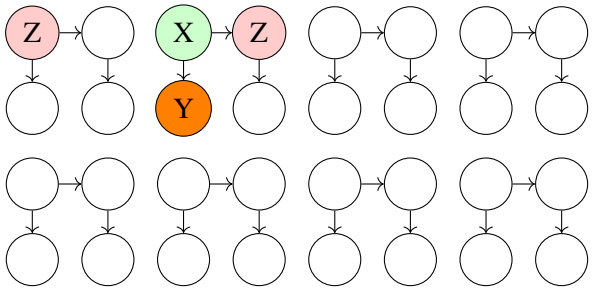



Fig. 2.5 Hybrid mapping with $2 \times 2$ cells showing the qubit operator representation of $\frac{i}{2}(a_6 - a_6^\dagger)$. Orange = target, Red = Parity set, Green = Update set

**Output:** *Pauli-strings corresponding to interaction edges, Enumerated square-lattice of qubits plus required ancillas, Preprocessing entanglement gates*

**Summary:** This algorithm essentially extends the Jordan-Wigner portion of the Hybrid mapping scheme in the same way AQM extends the Jordan-Wigner mapping. We add in an ancilla per tree and then perform entangling operators upon the roots of the trees and the ancillas such that particular stabiliser operations are constructed. We can then use these stabilisers to shorten non-local strings on the roots of the trees to a single operation on an ancilla.

**Algorithm:**

1. Perform steps 1-4 of the Hybrid mapping algorithm, but alternatively assign the *cell number* ($c_i$) in a S pattern (illustrated by Figure A.1.2 in Appendix A.1.2).

2. For every cell $c_i$, add one ancilla to the qubit enumeration scheme. This ancilla qubit is labelled $(c_i, n \times n + 1)$ and only needs to be connected to the root of the cell $(c_i, n \times n)$ and the ancillas of the neighbouring cells.

3. Prepare a stabiliser for each ancilla containing the string of Z gates involved in performing a fermionic operation between the roots of the current cell and the cell vertically above. (A detailed description of these stabilisers is given in Appendix A.1.2 Figure A.1.3). The stabilisers are created using the method described in [101] by applying the following series of entangling gates:

   (a) Apply a Hadamard gate to every ancilla

   (b) Entangle the edges of the rows by applying either of the gates controlled by the ancilla C shown in Figure 2.6 depending on the direction of the S-pattern on the row.

   (c) Move along each row applying either of the gates controlled by ancilla C shown in Figure 2.7 depending on the direction of the S-pattern on the row.

   This technique exploits the newly created stabiliser of the previously entangled ancilla to cancel out most of the gates required to entangle the next ancilla in the row.

4. For each interaction edge in your fermion interaction graph between $(c_1, f_1)$ and $(c_2, f_2)$ generate the Pauli-string representation:

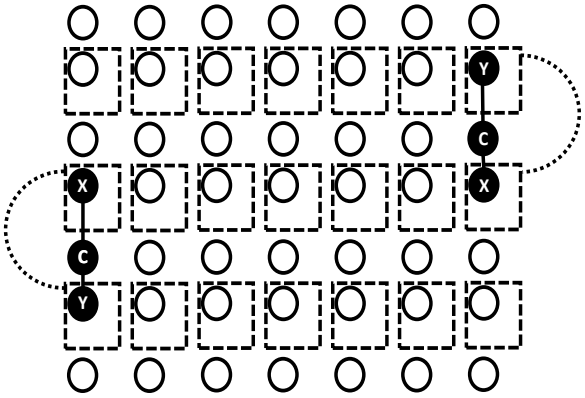



Fig. 2.6 Gates performed on edges of rows conditional upon the ancillas at the end of each row labeled with $C$. The dashed boxes represent the cells, whilst the circles inside them represent the roots of each cell, and the circles without boxes represent the ancillas.

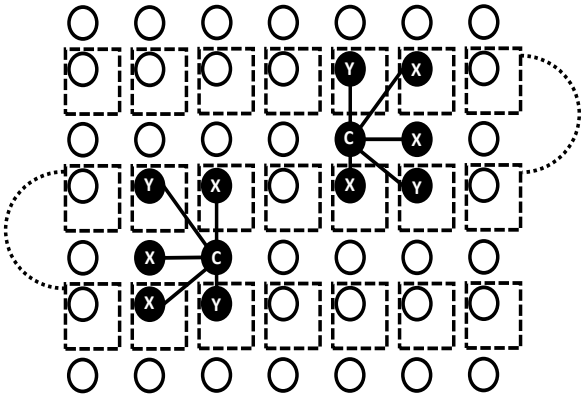



Fig. 2.7 Examples from the two (for odd/even rows) series of gates applied conditional upon the ancillas labeled with $C$. Each element in the series is identical to the previous but shifted one row to the left (odd rows) or right (even rows). The first element in each series is the 2nd rightmost (odd rows) and 2nd leftmost (even rows).

(a) Perform steps 5a)-c) of the Hybrid mapping algorithm

(b) We will only consider physical (even) fermionic operations, i.e. those which preserve the fermion number parity. For example:

$$a_1 a_2^\dagger + a_2 a_1^\dagger \to X_{U_1}\sigma_1^- Z_{P_1/F_1} X_{U_2}\sigma_2^+ Z_{P_2/F_2} + X_{U_2}\sigma_2^- Z_{P_2/F_2} X_{U_1}\sigma_1^+ Z_{P_1/F_1}$$
$$\to \frac{1}{2} \bigotimes_{j\in(U_1\oplus U_2)} X_j \bigotimes_{k\in(P_1/F_1\oplus P_2/F_2)} Z_k$$
$$\left( X_{(c_1,f_1)} \otimes X_{(c_2,f_2)} \bigotimes_{k\in F_1} Z_k \bigotimes_{j\in F_2} Z_j + Y_{(c_1,f_1)} \otimes Y_{(c_2,f_2)} \right)$$

In what follows we only need to consider how these operators act on the roots of the cells $(c_i, n\times n)$, as the stabiliser only deals with the cell roots. As $n\times n = 2^k$ each cell will contain a connected Fenwick tree, so each root will either be in the update set or be one of the targets. Therefore, each root is either acted on by $X_{(c_i,f_i)}$ or $X_{(c_t,f_t)} \otimes X_{(c_i,f_i)} \bigotimes_{k\in F_t} Z_k \bigotimes_{j\in F_i} Z_j + Y_{(c_t,f_t)} \otimes Y_{(c_i,f_i)}$.

(c) We need to modify every string so they preserve the stabilisers. For each stabiliser, add a Z gate (acting on the associated ancilla) to every string that anti-commutes with the stabiliser. We are also able to shorten long S pattern Pauli strings by multiplying by appropriate stabilisers. Figure 2.8 demonstrates how the Hybrid+ mapping modifies a Pauli-string representing a local fermionic operation in the Hybrid mapping scheme. Further examples are illustrated by Figures A.1.4 and A.1.5 in Appendix A.1.2.

If neither of the target fermions are the roots of their cell then the fermionic operation (on the level of cell roots) will simply be $XZ\ldots ZX$. However, if one of them is a root it will also include terms such as $XZ\ldots ZY$ and $YZ\ldots ZX$ (and potentially $YZ\ldots ZY$ if they are both roots).

## 2.1.2 Algorithm analysis

### 2.1.2.1 Theoretical Average Pauli-weight of $n\times n$ Hybrid on an $N\times N$ lattice

There are two primary contributions to the Pauli-weight of a fermionic operation in the Hybrid scheme:

1. Local (Bravyi-Kitaev type) gates acting on qubits within the target cells
2. Non-local (Jordan-Wigner type) gates acting on the roots of the cells between the two target cells in the Z-pattern

The first contribution scales with $O(\log(n^2))$, whereas the second contribution scales with $O(N)$.

We can analytically calculate the second contribution:

- Any interactions within a single cell make no contributions here

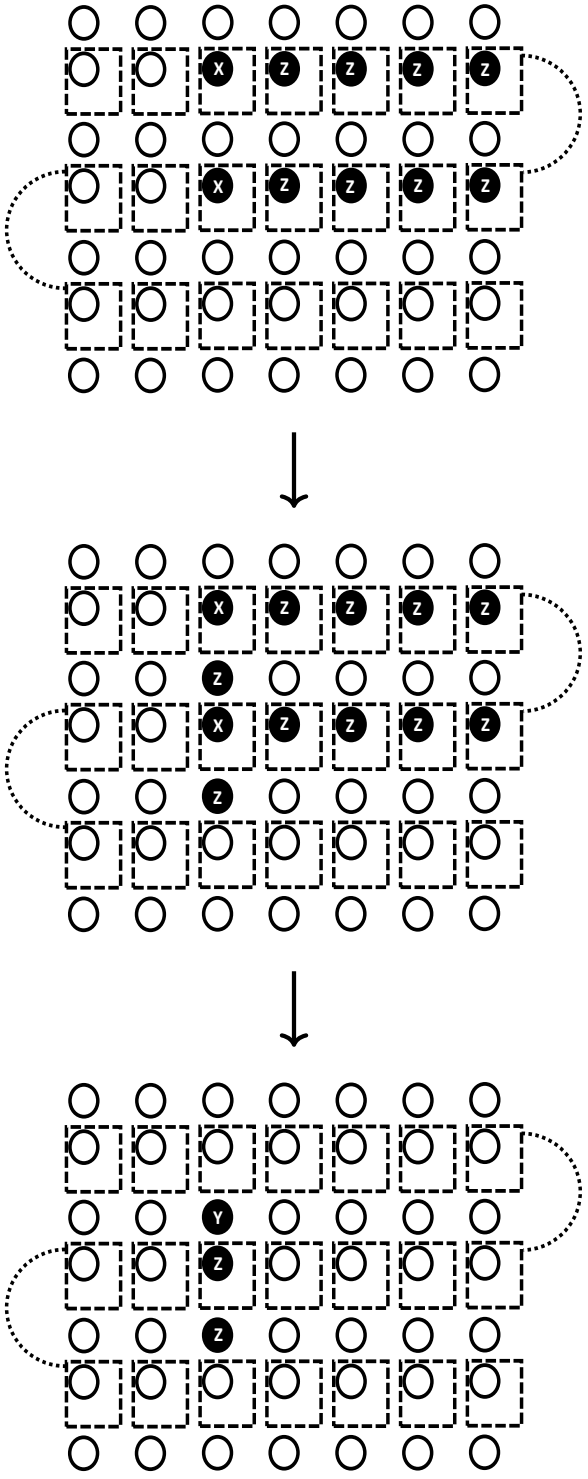



Fig. 2.8 Diagram showing how the Pauli-string representing a vertical hopping term (top) is modified by the Hybrid+ mapping algorithm. First, Z gates are added to all the ancillas with associated stabilisers that anti-commute with the string (middle). Then, a stabiliser shown in Figure A.1.3 is applied canceling out a large portion of the string to give the final local operation (bottom)

- Horizontal interaction terms between cells only contribute the root of one the target cells. However, as this root will already either be the target or in the update set we can ignore this contribution.
- Vertical interaction terms between qubits in different cells contribute $\frac{N}{n} - 1$. Therefore, the total contributions from each row is:

$$\sum_{i=1}^{N} \frac{N}{n} - 1 = N\left(\frac{N}{n} - 1\right)$$

  Therefore, as there are $\frac{N}{n}$ rows the total of the second contribution is:

$$\frac{N^2}{n}\left(\frac{N}{n} - 1\right)$$

  As there are $2N(N-1)$ interactions in the whole lattice this gives the following contribution to the average Pauli-weight:

$$\frac{N}{2(N-1)n}\left(\frac{N}{n} - 1\right)$$

Therefore, the average Pauli-weight of $n \times n$ Hybrid on an $N \times N$ lattice is:

$$\frac{N(N-n)}{2(N-1)n^2} + O(\log n)$$

and so for large lattices:

$$\frac{N}{2n^2} + O(\log n)$$

When $N \gg n$, the Hybrid mapping produces $n^2$ times smaller Pauli-strings than the Jordan-Wigner mapping.

#### 2.1.2.2 Theoretical Average Pauli-weight of $n \times n$ Hybrid+ on an $N \times N$ lattice

Similarly to the Hybrid mapping, there are two primary contributions to the Pauli-weight of a fermionic operation:

1. Local (Bravyi-Kitaev type) gates acting on qubits within the target cells
2. Local (AQM type) gates acting on ancilla qubits

The first contribution scales with $O(\log(n^2))$, whereas the second contribution scales with $O(\frac{1}{n})$ (as the ratio of inter-cell/intra-cell interactions decreases as $n$ increases). Therefore, as $N$ (and therefore the number of fermionic modes) increases the average Pauli-weight will remain constant.

#### 2.1.2.3 Varying impact of connectivity

The limitations of the underlying qubit architecture has a large impact on the performance of various mappings. In this section, we demonstrate the advantages our mappings give over existing schemes when

used on a quantum computer with the common square-lattice connectivity. The connectivity constraints impact mappings by requiring SWAP gates to perform entangled operations between physically non-local qubits. In our analysis, we compare the number of qubits acted upon by a Pauli or SWAP gate during a fermionic operation (the *range*). Below we consider the varying impact of connectivity upon different mappings.

**Jordan-Wigner Mapping**. The Jordan-Wigner mapping only requires a linear connectivity mapping, so it requires no additional gates to use on a square-lattice of qubits. However, it has been shown that an optimal choice of enumeration can give a constant improvement in the Pauli weight [27].

**Bravyi-Kitaev Mapping**. The Bravyi-Kitaev mapping requires a very high level of connectivity. In order to avoid any additional gates one must be able to embed an $\log N$ degree graph (the Fenwick tree) within the qubit architecture. As a square-lattice of connectivity has degree 4 it is not possible to embed a Bravyi-Kitaev mapping of more than 16 fermionic modes. Therefore, for Bravyi-Kitaev mappings on larger systems neighbouring Fenwick tree nodes will not be connected, so additional SWAP gates are required to perform entangled gates on these qubits.

We found that simulating these additional SWAP gates drastically reduced the performance of the Bravyi-Kitaev mapping. This is theoretically supported by considering the scaling of the largest range interactions. Consider the fermionic operation between the fermions corresponding to qubits which are physically located at opposite ends of a row (or column), this operation must include these qubits as they are the targets. To connect just these two qubits at least $N$ qubits must be involved in the operation. Our unoptimized Z-pattern enumeration for the Bravyi-Kitaev mapping includes non-local interactions such as these, and hence the average range scales with $O(N)$. It is feasible that an optimal Bravyi-Kitaev enumeration could be found that scales more favourably. However, this is a very hard problem as the number of possible enumeration schemes grows factorially with lattice size and there is no known solution.

**Hybrid mapping**. The main strength of this mapping is its performance in the real-world scenario of a square-lattice connectivity. It combines the Jordan-Wigner mappings connectivity flexibility with the Bravyi-Kitaev mappings increased locality. The average interaction qubit count contribution from within the Bravyi-Kitaev cells scales as above with $O(n)$. While the Pauli operations on the roots of the cells along the S-pattern are joined up with SWAP gates making the contribution per row from vertical interactions:

$$\sum_{i=1}^{N}(N-1) = N(N-1)$$

Therefore, as there are $\frac{N}{n}$ rows and $2N(N-1)$ interactions in the whole lattice, the contribution to the average range is:

$$\frac{N}{2n}$$

Therefore, the average range scales with $O(n+\frac{N}{2n})$. So for $N \gg n$, this mapping has $n$ times lower average range than both the Bravyi-Kitaev and Jordan-Wigner mappings.

**Hybrid+ mapping**. This algorithm requires a slightly higher level of connectivity. Our formulation has a second square-lattice of ancilla qubits connected to the original at the roots of the cells. It could also

be formulated by instead choosing one qubit in each cell of the original square lattice to be an ancilla. However, we chose our formulation as it produces a lower range, is simpler to consider, and still is a realistic qubit architecture.

The Hybrid+ mapping retains a constant average Pauli-weight when considering the limitations of connectivity. This is due to the only rising contribution coming from the Bravyi-Kitaev mapping within the target cells which should scale with roughly $O(n)$, and for a given mapping $n$ is constant.

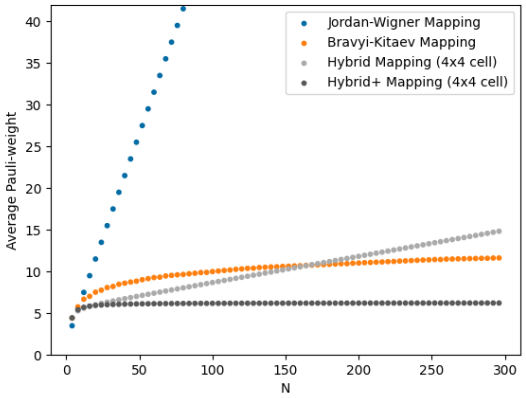



Fig. 2.9 The average Pauli-weight of fermionic operations on an $N \times N$ lattice against $N$ when using a given mapping.

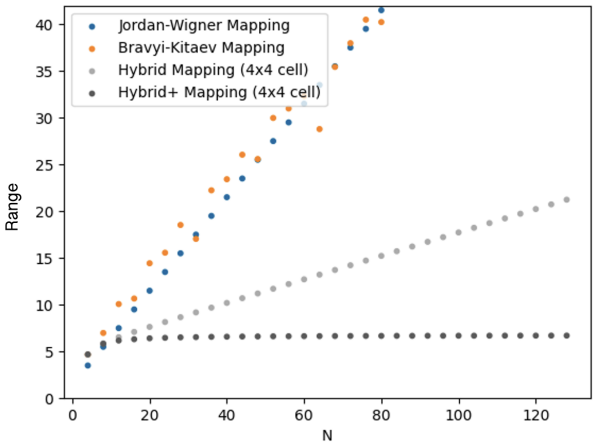



Fig. 2.10 The average range of each fermionic operation on an $N \times N$ lattice against $N$ when using a given mapping in the context of a square-lattice qubit connectivity.

#### 2.1.2.4 Numerical Analysis

We have considered the locality of each algorithm by computing the average range on a $N \times N$ square lattice. When investigating the case of a $4 \times 4$ cell, we found that the theoretical predictions held; the

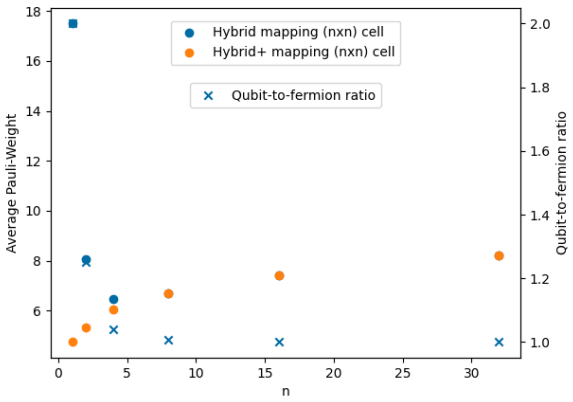



Fig. 2.11 The average Pauli-weight of a fermionic operation on an $32 \times 32$ lattice when using an $n \times n$ Hybrid/Hybrid+ mapping against $n$. The crosses denote the qubit-to-fermion ratio of the $n \times n$ Hybrid+ mapping for each choice of $n$.

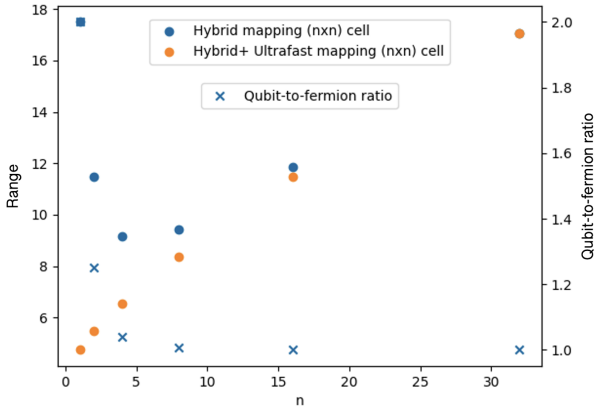



Fig. 2.12 The average range of each fermionic operation on an $32 \times 32$ lattice when using an $n \times n$ Hybrid/Hybrid+ mapping against $n$ in the context of a square-lattice qubit connectivity.The crosses denote the qubit-to-fermion ratio of the $n \times n$ Hybrid+ mapping for each choice of $n$.

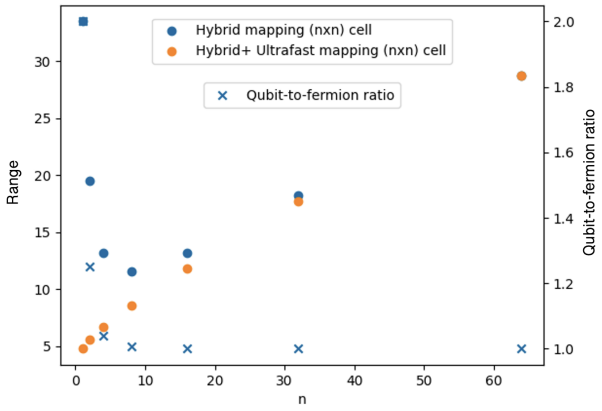



Fig. 2.13 The average range of each fermionic operation on an $64 \times 64$ lattice when using an $n \times n$ Hybrid/Hybrid+ mapping against $n$ in the context of a square-lattice qubit connectivity for $n = \{1, 2, 4, 8, 16, 32, 64\}$. These results include qubits solely acted on by SWAP gates. The crosses denote the qubit-to-fermion ratio of the $n \times n$ Hybrid+ mapping for each choice of $n$.

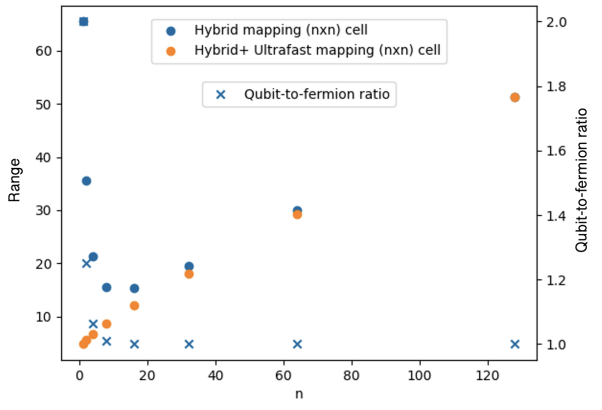



Fig. 2.14 The average range of each fermionic operation on an $128 \times 128$ lattice when using an $n \times n$ Hybrid/Hybrid+ mapping against $n$ in the context of a square-lattice qubit connectivity for $n = \{1, 2, 4, 8, 16, 32, 64, 128\}$. These results include qubits solely acted on by SWAP gates. The crosses denote the qubit-to-fermion ratio of the $n \times n$ Hybrid+ mapping for each choice of $n$.

average interaction qubit count scaled with $\frac{N}{32}$ for the Hybrid algorithm and asymptotically approached a constant for the Hybrid+ algorithm (as illustrated in Figure 2.9).

We performed the same analysis in the context of an underlying square lattice connectivity. This required calculating the minimum number of SWAP gates required to connect all the qubits involved in the operation. We approximated the solution to this Steiner Tree problem using the SCIP-JACK library outlined in Appendix A.1.1. Figure 2.10 shows that the theoretical predictions hold with the average range increasing with $\frac{N}{8}$ for the Hybrid algorithm and tending to a constant for the Hybrid+ algorithm.

The largest difference evident is in the Bravyi-Kitaev mapping. When including SWAP gates, the average range involved appears to be scaling linearly rather than logarithmically. In our calculations we used a trivial Z-pattern enumeration scheme for the Bravyi-Kitaev mapping, so it is possible the performance could be improved by an optimal choice of enumeration scheme.

Therefore in the context of an underlying square lattice connectivity the $4 \times 4$ Hybrid mapping outperforms both the Jordan-Wigner and Bravyi-Kitaev mapping by a factor of 4 . Furthermore, even with all-to-all connectivity it still outperforms the Jordan-Wigner mapping by a factor of 16, and beats the Bravyi-Kitaev mapping for lattices smaller than $164 \times 164$.

We have investigated which cell sizes give optimal performance. The behaviour shown in Figure 2.11 and 2.12 matches with our theoretical predictions. For the Hybrid mapping, in the regime of $n \ll N$ the average range scales with $O(\frac{1}{2n^2})$ (for fixed $N$). However, this is quickly dominated by the $O(\log n)$ growth of the Bravyi-Kitaev mapping within each cell. Therefore, as the logarithm dominates for $K\frac{N}{2n^2} < \log n$ (for constant $K$), we hypothesis that the optimal choice of $n$ will scale with roughly $O(\sqrt{N})$. The trend of increasing optimal choice of $n$ with lattice size is demonstrated by the increasing optimal cell size from $4 \times 4$ to $8 \times 8$ to $16 \times 16$ when the lattice size is increased from $32 \times 32$ (Figure 2.12) to $64 \times 64$ (Figure 2.13) to $128 \times 128$ (Figure 2.14).

For the Hybrid+ mapping, there is no $O(\frac{1}{2n^2})$ regime as there are no non-local gates acting on the roots. Therefore, the behaviour is dominated by the Bravyi-Kitaev mapping within the cell. This is reflected in the $O(\log n)$ growth of average range. However, the optimal cell size is not simply the smallest (equivalent to AQM [101]). The main benefit of this mapping is in offering a direct trade-off between number of ancillas and range. As the current minimum qubit-to-fermion ratio achieved by a local mapping is 1.25 [25], we have analysed the next possible cell size of $4 \times 4$ which achieves a qubit-to-fermion ratio of 1.0625. However, using larger cell sizes requires far fewer ancilla qubits. For example, an $8 \times 8$ Hybrid+ mapping requires 94% fewer ancillas than the super-compact mapping scheme [25].

### 2.1.3 Discussion

We have presented two novel mappings: the Hybrid mapping achieves the shortest circuits of all known mappings without ancillas. The margin of improvement is significant. As demonstrated in our simulations of a real-world qubit architecture, it generates 75% lower range interactions than the Jordan-Wigner and Bravyi-Kitaev mappings. Additionally, as the lattice sizes increases the optimum cell size grows and the performance of the Hybrid mapping further improves.

For calculations on current or near-term quantum computers the Hybrid mapping demonstrates an improvement over the Bravyi-Kitaev and Jordan-Wigner mappings even with all-to-all connectivity. This holds for lattices up to $164 \times 164$ (Figure 2.9). Lattices larger than this would require more than 27000 qubits to simulate, which is much larger than the current largest quantum computer (433 qubits [39]). Therefore, the Hybrid mapping will likely outperform the Bravyi-Kitaev and Jordan-Wigner mappings on real devices for the foreseeable future.

The Hybrid+ mapping demonstrates a remarkable improvement on all existing local mapping schemes. It achieves similar levels of locality to the Derby-Klassen [31], Super-compact [25], and AQM [101] type mappings but expends only a small fraction of the ancillas in comparison. Our example of a $4 \times 4$ Hybrid+ mapping required 75% less ancillas than the local (super-compact) mapping with the lowest known ancilla requirements.

The Hybrid+ mapping also provides a direct and simple trade-off between locality and ancilla qubits. Increasing the cell size $n$ increases the range linearly whilst the number of ancillas required decreases with $O(\frac{1}{n^2})$.

The approach we have taken in constructing both of these mappings need not be restricted to square-lattice qubit architectures. It should be possible to reproduce similar mappings for any architecture and corresponding fermionic interaction graph that admit a cellular decomposition.

We have considered a fermionic interaction graph which coincides with our architecture. It would be interesting to explore mapping an arbitrary fermionic interaction graph onto a given architecture using a similar Hybrid construct of connected Bravyi-Kitaev cells. In particular, we would like to understand how this mapping would translate into 3D, and whether this would require a 3D qubit connectivity or if cubic cells can be mapped to a planar grid efficiently.

In our analysis, we found connectivity constraints significantly hampered the Bravyi-Kitaev type terms in the mapping. Further work could explore this phenomenon in more detail. It is possible that an optimal choice of fermionic enumerations following the optimization techniques of [27] could significantly reduce the impact of the connectivity constraints.

A lesser-known but equally remarkable role of fermion-to-qubit mappings is their role as duality transformations. The Jordan-Wigner transform [61] can be regarded as an example of such a duality transform [51]. It maps spin Hamiltonians (such as XXZ chain) to simple fermionic systems which can be investigated by classical quantum chemistry techniques or by quantum means [86, 19]. Our two new families of fermion-to-qubit mappings could be interpreted as efficient duality transformations. Therefore, an interesting application of these hybrid mappings is the investigation of complex Hamiltonians.

## 2.2 Local Jordan-Wigner transformations and topological sectors on the torus

Quantum wavefunctions are described on a projective space, and this allows a system of indistinguishable particles to be described by anti-symmetric wavefunctions, a.k.a. fermions. By imposing a 1-dimensional ordering to a collection of fermions, Jordan and Wigner [61] demonstrated how to endow the fermions with a tensor product structure. This was done by introducing a duality mapping between the fermions and a system of spin $1/2$ particles (i.e. qubits). However, this original mapping only preserves locality for 1-D systems. The concept of flux attachment [58, 71, 46, 54, 37] suggested the existence of higher-dimensional analogues of this Jordan-Wigner transformation preserving locality in arbitrary dimensions and in the thermodynamic limit; the first explicit constructions of such a mapping in 2 spatial dimensions was obtained in [15, 111, 8]. A system of fermions located on the vertices of a square lattice was mapped to a system of qubits living on the edges of that graph. The fact that this construction yields twice as many qubits as fermionic modes was compensated by the fact that the new system exhibits gauge conditions on every plaquette. This work was followed by many other papers [97, 109, 48, 113, 50, 99, 102, 60, 7, 59, 32, 28, 82, 26, 87] that improved the qubit-to-fermion ratio and/or the maximum support in the spin representation of a single fermionic hopping term. The modern point of view is that the fermion to qubit mapping is a duality transformation obtained by gauging the global $Z_2$ parity symmetry of the fermions [24]; in such a mapping, the fermion matter can then be disentangled by a local fermionic quantum circuit [116]. However, many questions remained open: it was not clear how to modify the mappings when dealing with odd charge sectors especially when dealing with periodic and twisted boundary conditions on a torus. In this paper, we show how those problems can be solved by making use of the formalism of graded tensor networks [18, 29] and more specifically of projected entangled pair operators which explicitly encode the gauge transformations [45, 100].

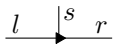
This section is organized as follows. We start by defining the graded projected entangled pair operator (PEPO) for the fermion to qubit mapping on arbitrary graphs. Subsequently, we revisit the 1-D Jordan-Wigner transformation for a (anti-)periodic ring of fermions in the even and odd charge sectors in terms of matrix product operators (MPOs). Next, we show how this mapping can readily be extended to the 2-dimensional torus, and pay special attention to the intriguing interplay between charge sectors and boundary conditions. Controlling the possible boundary conditions turns out to be important for state of the art simulations of e.g. the Hubbard model [114]. Hence our results can be utilised to improve fermionic quantum simulation by reducing the finite size effect through the introduction of (anti-)periodic boundary conditions.

The larger motivation behind our work is to reduce the entanglement of fermionic ground states by mapping them to a spin setting, following recent work [69]. Our PEPO introduces an extensive number of symmetries on the spin side, resulting in a highly degenerate spin ground state corresponding to a unique fermionic ground state. Within this degenerate space, a state might exist with significantly lower entanglement that breaks all the introduced symmetries. This lower-entanglement state is more amenable

to tensor network methods, enabling efficient simulation of highly entangled fermionic systems through our construction.

*Construction*—Given an arbitrary graph $G = (V,E)$, we construct our PEPO by placing fermionic $Z_2$ graded $(d+1)$-degree vertex tensors [18] at every $d$-degree vertex of $G$ and a graded GHZ tensor on every edge of $G$.

The GHZ tensor is defined as:




$$:= \sum_c \langle c|_s (c|_l (c|_r .$$

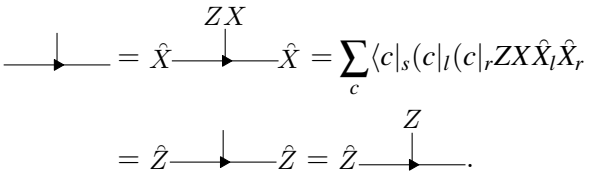
where $l$ and $r$ label the left and right virtual fermionic legs of the tensor and $s$ labels the physical spin leg. Every tensor must be drawn with an arrow which dictates what is left and right, but as we work on arbitrary graphs, no spin structure is needed. Curved brackets will be used to denote fermionic degrees of freedom, while the usual bra-ket notation will be used for the qubit degrees of freedom. We can consider curved brackets to be defined in terms of creation operators acting on the vacuum with $|1) = c^\dagger|\Omega), |0) = |\Omega)$ and hence:

$$|a)|b) = (-1)^{ab}|b)|a), \quad \mathrm{Tr}(|a)(b|) = (-1)^{ab}.$$

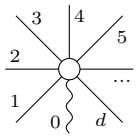
Throughout this work we refer to the fermionic $\hat{X}$ and $\hat{Z}$ operators which we define as:

$$\hat{X} = \sum_a |a+1)(a|, \quad \hat{Z} = \sum_a (-1)^a |a)(a|.$$

We study the symmetries given by applying products of these operators to the GHZ tensors:



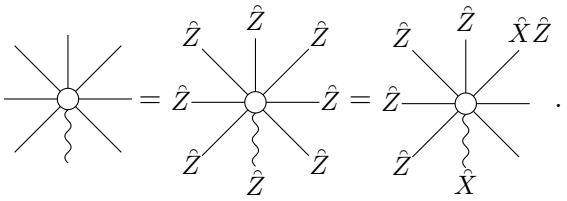

When writing symmetries in this graphical notation, we require that the operators applied are ordered with the same internal ordering of the legs which they apply to. The vertex tensor is defined as:




$$:= \sum_{\substack{\{a_i\} \\ \sum_i a_i = 0 \,(\mathrm{mod}\, 2)}} |a_d) \ldots |a_1)|a_0),$$

where $a_0$ labels the physical fermionic leg of the tensor and $a_1, \ldots, a_d$ label the virtual fermionic legs which will be contracted with the GHZ tensor; the ordering is important and must be specified on every vertex. In this work we always using the ordering of clockwise starting at the physical fermionic leg. Any choice of orderings will be valid, however, changing the internal ordering will lead to different mappings.

With this in mind we consider the symmetries of the vertex tensors:




The constructed PEPO will map fermionic operators acting on the vertices of $G$ to spin operators acting on the edges of $G$. We represent this (before contraction of the virtual legs) as $\langle s|\,(E||V_v)|V_p)$ where $\langle s|$ are the physical spins, $(E|$ are the virtual fermions on the GHZ tensors, $|V_v)$ are the virtual fermions on the vertex tensors and $|V_p)$ are the physical fermions. This mapping will be local in the sense that even fermionic hopping terms acting along edges of the graph $G$ will map to spin operators with a maximum Pauli weight $2\Delta-1$ (where $\Delta$ is the maximum degree of $G$).

As all tensors involved in the PEPO have an even parity, a tensor network diagram uniquely specifies the mapping, as long as arrows are put on all edges and an internal ordering of vertex tensors is chosen. Note that defining a fermionic operator (e.g. $\hat{O}=a_j^\dagger a_i$) requires a linear ordering for the Hilbert space it acts upon (e.g. $j<i$). A change to this ordering (e.g. to $i<j$) incurs a global minus sign on the fermionic operator giving a new definition of the same operator ($\hat{O}=-a_j^\dagger a_i$) acting on a different Hilbert space. As our mapping is not affected by the choice of linear ordering it incurs no sign change upon reordering (it will always map $a_j^\dagger a_i$ to $O$). Therefore, whether $\hat{O}$ is mapped to $+O$ or $-O$ depends on the linear ordering chosen for the fermionic Hilbert space being mapped.

Using the symmetry properties of our tensors before contraction, fermionic operators acting on the physical fermionic degrees of freedom ($O_{pf}$) are transformed to to fermionic operators acting on the virtual fermionic legs of our vertex tensors ($O_{vf(ver)}$). This is equivalent to acting with the same operators on the virtual fermionic legs of our edge tensors ($O_{vf(edg)}$), which then transform them to spin operators acting on the physical spin degrees of freedom ($O_s$):

$$\begin{aligned}
O_{pf}\,\langle s|\,(E||V_v)|V_p) &= \langle s|\,(E|O_{pf}|V_v)|V_p)\\
&= \langle s|\,(E|O_{vf(ver)}|V_v)|V_p)\\
&= \langle s|\,(E|O_{vf(edg)}|V_v)|V_p)\\
&= \langle s|\,O_s(E||V_v)|V_p)\\
&= \langle s|\,(E||V_v)|V_p)O_s.
\end{aligned}$$

The symmetries of our tensors reveal that the theory at the side of the spins is a gauge theory, with a gauge condition for every possible cycle in the graph. The number of independent gauge conditions is given by the circuit rank $r$ of $G$ (the number of independent cycles in $G$). As $r=|E|-|V|+1$, this mapping is an isomorphism from the even fermionic sector to the $+1$ joint eigenspace of the gauge conditions on the spin side. On a torus two of these independent gauge conditions correspond to non-contractible loops;

these gauge conditions can be lifted and interpreted as charge sectors, and those different charge sectors will correspond to (twisted) boundary conditions on the fermion side.

To admit a mapping of odd fermionic states we add a single odd tensor defect (fermionic $\hat{X}$ operator) to a virtual leg of the PEPO. This fermionic charge defect can be placed anywhere, and its presence can be understood as a boundary condition for the dual spin theory: a manifestation of the effect that dualities switch boundary conditions with charge sectors.

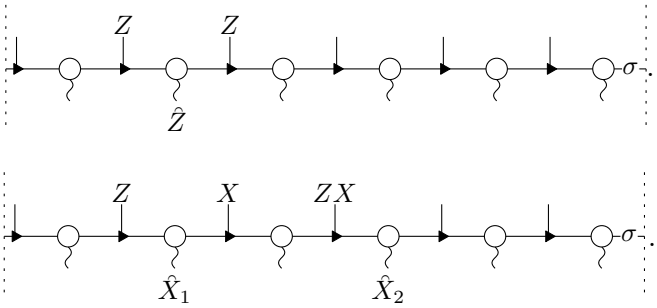
A single fermionic creation/annihilation operator will be mapped to a string of spins terminating at the odd defect. If strings of two different operators converge at any point before reaching the defect they will anticommute. If two strings never converge but instead approach the defect from opposite directions, then they will necessarily anticommute on the closest spin to the defect.

Hence, we produce the correct anti-commutation relations for the fermionic algebra.

It can be seen from the symmetries of our tensors that our construction on a general graph without a defect is an explicit PEPO representation of the BKSF mapping [15]. Therefore, due to the equivalence of 2D fermionic encodings shown by [26] it is possible to reconstruct any existing 2D fermion-to-qubit mapping through a particular choice of $G$, internal orderings, single qubit gates and majorana repairing. Further the inclusion of the $\hat{X}$ defect allows for the representation of tree-based mappings such as the Bravyi-Kitaev Fast Mapping [15]. We will expand on this equivalence in future work.

*1D Construction*—On a 1D cycle graph, our construction turns out to be equivalent to the Jordan-Wigner mapping composed with the Kramers–Wannier duality. We will use this as an example to illustrate our formalism. Further insight into how 1D boundary conditions map charge sectors can be found in Appendix D of [100].

The 1D PEPO is an MPO formed of interleaved GHZ tensors and 3-degree vertex tensors. The mappings of fermionic $\hat{Z}$ and $\hat{X}\hat{X}$ operations can be inferred from the invariance of the MPO under the following operations:




In this particular example (with $\sigma$ boundary condition), a fermionic $\hat{Z}_i$ operator is mapped to a $Z_{i-1/2}Z_{i+1/2}$ term on the spin side, and a fermionic $\hat{X}_{i-1}I_i\hat{X}_{i+1}$ to $Z_{i-3/2}X_{i-1/2}(XZ)_{i+1/2}$. Different choices of boundary condition will map different topological sectors. These sectors are characterised by the charge sector under the global symmetries of each side, and the twists required for translational symmetries. Charge sectors are the eigenspaces of a symmetry operator. Twists are the modifications required to either side of a PEPO upon translation in order to recover the same PEPO.

The charge sector mapped by a boundary condition is determined by the sign produced when the MPO absorbs the given symmetry. As in the example below, it is important to account for the signs due to moving any unpaired fermionic operators at the boundaries next to each other and into the correct order to be absorbed by the vertex symmetries without additional signs.

$$\begin{aligned}
&\langle s|\,(E|\sigma_n|V_v)|V_p\rangle \prod_i X_i = \langle s| \prod_i X_i (E|\sigma_n|V_v)|V_p\rangle \\
&= \langle s|\,(E|Y_1X_2Y_3\ldots X_{n-2}Y_{n-1}X_n\sigma_n|V_v)|V_p\rangle \\
&= \langle s|\,(E|Y_1X_n\sigma_n|V_v)|V_p\rangle \\
&= \begin{cases} -\langle s|\,(E|X_n\sigma_nY_1|V_v)|V_p\rangle & \sigma \text{ is even} \\ \langle s|\,(E|X_n\sigma_nY_1|V_v)|V_p\rangle & \sigma \text{ is odd} \end{cases} \\
&= \begin{cases} -\langle s|\,(E|\sigma_n|V_v)|V_p\rangle & \sigma = I \\ \langle s|\,(E|\sigma_n|V_v)|V_p\rangle & \sigma = Z \\ \langle s|\,(E|\sigma_n|V_v)|V_p\rangle & \sigma = X \\ -\langle s|\,(E|\sigma_n|V_v)|V_p\rangle & \sigma = Y \end{cases}
\end{aligned}$$

The twist produced by a boundary condition under a translation is calculated by shifting over each physical leg above and below the MPO, and considering the necessary operators produced on the physical legs by moving the boundary condition to the new boundary. For example, translating the $\hat{Z}$ boundary condition produces a $\hat{Z}$ twist on a fermionic physical leg:

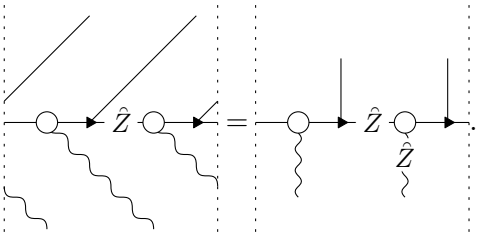


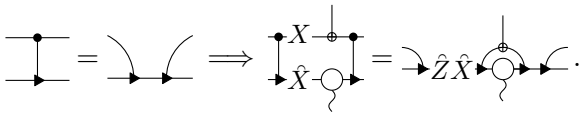

Table 2.2 illustrates how the topological sectors of the 1D cyclical graph are determined and mapped by different boundary conditions. By treating these boundary conditions as additional quantum degrees of freedom, we are able to recover a completely unitary transformation from fermions to spins. If we did not consider boundary conditions we would only have an isometry between the periodic even fermionic space and the periodic $-1$ spin eigenspace of $\prod_i X_i$, rather than mapping the entire fermionic Hilbert space to the entire spin Hilbert space.

We may compose our MPO with the Kramers-Wannier MPO in order to recover the canonical Jordan-Wigner transformation. The composition will necessarily evaluate to zero if the charge sectors are not matched. However, mismatched twists will give a non-zero MPO, therefore, it is necessary to chose the unique KW boundary condition which matches the twist and charge sector as seen in Table 2.2. We can

| Fermions | | | Spins | | | Dual Spins | | Unified |
|---|---|---|---|---|---|---|---|---|
| $\prod_i \hat{Z}_i$ | Twist | BC 1 | $\prod_i X_i$ | Twist | BC 2 | $\prod_i Z_i$ | Twist | BC |
| $+1$ | $I$ | $I$ | $-1$ | $I$ | $Z$ | $+1$ | $Z$ | $\hat{Z}$ |
| $+1$ | $\hat{Z}$ | $\hat{Z}$ | $+1$ | $I$ | $I$ | $+1$ | $I$ | $\hat{Z}$ |
| $-1$ | $I$ | $\hat{X}$ | $+1$ | $X$ | $X$ | $-1$ | $I$ | $\hat{Z}\hat{X}$ |
| $-1$ | $\hat{Z}$ | $\hat{Z}\hat{X}$ | $-1$ | $X$ | $ZX$ | $-1$ | $Z$ | $\hat{Z}\hat{X}$ |

Table 2.2 Mapping of topological sectors of fermions to spins on the chain dependent on the choice of boundary condition (BC 1) for the duality MPO. These sectors can then be mapped again to the dual spins by the application of a Kramers-Wannier MPO with a suitable choice of boundary condition (BC 2). The composed mapping from fermions to dual spins is equivalent to the canonical Jordan-Wigner mapping with one single boundary condition (Unified BC).

then combine these two boundary conditions as follows:

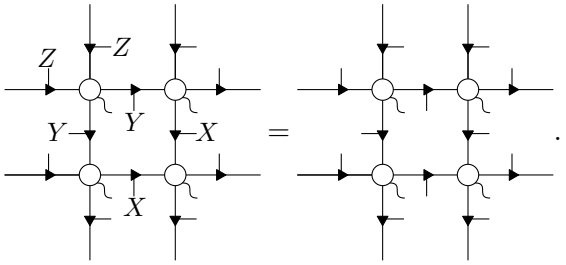



Hence, the $\hat{Z}$ and $\hat{Z}\hat{X}$ boundary conditions map both the periodic and anti-periodic boundary conditions for the even and odd sectors respectively. The $I$ and $\hat{X}$ boundary conditions evaluate to 0 (as they correspond to mismatched charge sectors between the combined MPOs), so we can simply add them to their non-zero counterparts ($\hat{Z}$ and $\hat{Z}\hat{X}$) to obtain rank 1 boundary conditions $|0)(0|$ and $|1)(0|$. By summing these two MPOs we produce a unified MPO with boundary $(|0)+|1))(0|$ giving a unitary mapping from fermions to spins. This clarifies the mystery of why one and the same JW-mapping can deal with all four topological sectors - which is very unusual and unexpected for a duality transformation.

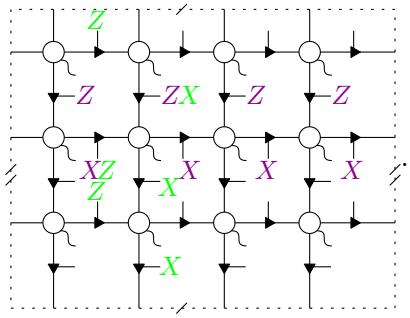
We note that using our formalism the boundary conditions are sensitive to their placement in the MPO. For example, in 1D the $\hat{Y}$ boundary condition to the right of a 3-degree vertex tensor is similar to a $\hat{X}$ boundary condition to the left of a 3-degree vertex tensor. For consistency we always refer to the boundary condition as acting to the left of the 3-degree vertex tensor.

*2D Construction*—For any planar graph $G$ on an orientable manifold, there will be a natural choice for the orientations of the vertex tensors. In this section we exemplify our mapping using the square lattice. In the bulk, our PEPO is similar to the one constructed in [100] and consists out of two simple tensors, a GHZ tensor and a 5-degree vertex tensor, arranged on the edges and vertices of a square lattice. On every

plaquette, the symmetries impose a gauge condition upon the spin system:




The number of vertices is exactly equal to the number of edges minus the number of plaquettes, this is the complete set of gauge conditions, if it were not for the fact that two of them are not independent. Of course, the two missing gauge conditions are the ones winding around the torus. These are given by loops of spin operators along the horizontal ($X_H$, magenta) and vertical ($X_V$, green) directions:



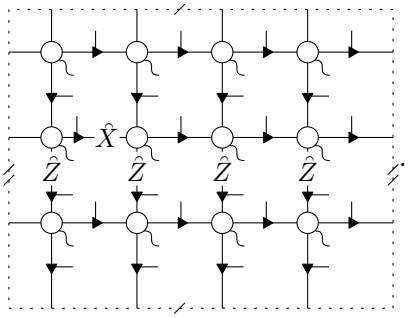

From the point of view of the fermion side, changing the charge sector on the spin side is equivalent to adding twisted boundary conditions. These boundary conditions are strings of fermionic $\hat{Z}$ operators lying along the virtual legs of the PEPO in the horizontal ($\hat{Z}_H$) and vertical ($\hat{Z}_V$) directions respectively. Table 2.3 illustrates how these boundary conditions are mapped on a torus. It is interesting to see how the four boundary conditions and two charge sectors are mapped to two boundary conditions and four charge sectors.

Boundary conditions are sensitive to their position in the tensor network, so for simplicity we will only place the $X$ defect on horizontal virtual legs to the right of the GHZ tensor. We are free to choose which non-contractible loops are $X_H$ and $X_V$. We have chosen them such that they intersect with the $\hat{X}$ defect if present, so the table matches the results for the 1D case. However, it would be equally valid to choose them to avoid the $\hat{X}$ defect which would make the spin charge sectors symmetrical for the even and odd fermionic sectors.

| Fermions | | | | Spins | |
|---|---|---|---|---|---|
| $\prod_i \hat{Z}_i$ | H. Twist | V. Twist | BC | $X_H X_V$ | H./V. Twist |
| $+1$ | $I$ | $I$ | $I$ | $-1-1$ | $I$ |
| $+1$ | $I$ | $\hat{Z}_{p_H}$ | $\hat{Z}_H$ | $-1+1$ | $I$ |
| $+1$ | $\hat{Z}_{p_V}$ | $I$ | $\hat{Z}_V$ | $+1-1$ | $I$ |
| $+1$ | $\hat{Z}_{p_V}$ | $\hat{Z}_{p_H}$ | $\hat{Z}_V\hat{Z}_H$ | $+1+1$ | $I$ |
| $-1$ | $I$ | $I$ | $\hat{X}$ | $+1+1$ | $Z_v X_h / Z_h X_v$ |
| $-1$ | $I$ | $\hat{Z}_{p_H}$ | $\hat{Z}_H\hat{X}$ | $+1-1$ | $Z_v X_h / Z_h X_v$ |
| $-1$ | $\hat{Z}_{p_V}$ | $I$ | $\hat{Z}_V\hat{X}$ | $-1+1$ | $Z_v X_h / Z_h X_v$ |
| $-1$ | $\hat{Z}_{p_V}$ | $\hat{Z}_{p_H}$ | $\hat{Z}_H\hat{Z}_V\hat{X}$ | $-1-1$ | $Z_v X_h / Z_h X_v$ |

Table 2.3 Table illustrating how different boundary conditions of the PEPO on the torus map different charge sectors and produce different twists. The $\hat{Z}_{p_V}$ and $\hat{Z}_{p_H}$ twists are vertical and horizontal loops of $\hat{Z}$ operators on the physical fermions. The $Z_v X_h / Z_h X_v$ twist (with subscript $v/h$ indicating whether the operator is applied on a vertical or horizontal leg) constitutes a single unit cell of $X_H$ or $X_V$ depending on whether the twist was taken in the horizontal or vertical direction respectively.

Below is an example of a PEPO with a $\hat{Z}_H$ boundary condition and an $\hat{X}$ defect, which will map between the odd fermionic algebra and the $(-1,+1)$ eigenspace of $(X_H, X_V)$:




We have made the choice of a square lattice to illustrate the mapping of topological sectors. This choice is not necessary to reproduce the results of Table 2.3, and suitable definitions of boundary conditions/charge sectors/twists on other 2D lattices (e.g. Kagome or honeycomb) will produce identical results.

*Conclusion*—We have shown how boundary conditions on a torus relate to various topological charge sectors. By treating those boundary degrees of freedom as extra quantum degrees of freedom, this allows us to construct a unitary operator (as opposed to an isometry) implementing the duality transform, demonstrating that the higher dimensional JW transformation completely preserves the spectrum of this bigger theory. This is completely in line with the results in [70, 67, 68] for all possible dualities defined in terms of categorical symmetries.

Further, we have demonstrated a tensor network approach to constructing arbitrary fermion to qubit encodings. The resulting spin system is a gauge theory with a very particular type of gauge constraints. We conjecture that such gauge theories in two dimensions can only be "ungauged" with graded tensor network intertwiners such as the PEPO we have described. This highlights the difference between the 1- and the 2-D case, as fermions seem to be indispenable in the latter whereas in the 1D case we can easily bosonise the virtual legs of the MPO.

We expect our methodology could be extended to higher dimensions in line with other systematic generalisations [23, 22]. Higher dimensions would lead to even more possible boundary conditions. For example, in 3D we would expect our mapping between charge sectors and boundary conditions to be between sheets of Paulis and lines of Paulis.

*End Matter on Unitarity*—In this section we discuss how our mappings are made unitary by treating the defect boundary condition as an additional degree of freedom.

The 1D MPO is unitary as the Fock basis is mapped one-to-one onto a spin basis. We can represent the fermionic state $|f_0 f_1 \dots f_n)$ in the stabiliser formalism:

$$\{I, (-1)^{f_0}\hat{Z}_0, \dots, (-1)^{f_{n-1}}\hat{Z}_{n-1}, (-1)^{f_n}\hat{Z}_n\}.$$

The MPO with $d$ odd defects maps this to:

$$\{I, (-1)^{d+1}\textstyle\prod_i X_i, (-1)^{f_0}Z_0Z_1, \dots, (-1)^{f_{n-1}}Z_{n-1}Z_n, (-1)^{f_n+d}Z_nZ_0\}, \tag{2.1}$$

where the additional $(-1)^{d+1}\prod_i X_i$ stabiliser comes from the degenerate mapping of the identity due to the symmetries of the MPO. Multiplying all the $Z_iZ_{i+1}$ stabilisers together gives $(-1)^{\sum_i^n f_n+d}I$. As $-I$ only stabilises the empty set, there is a unique choice of boundary condition which gives a non-empty result, simplifying (2.1) to:

$$\{I, (-1)^{\sum_i^n f_i+1}\textstyle\prod_i X_i, (-1)^{f_0}Z_0Z_1, \dots, (-1)^{f_{n-1}}Z_{n-1}Z_n\}.$$

These $n$ independent generators uniquely specify an $n$ spin state parameterised by $(\sum_i^n f_i + 1, f_0, \dots, f_{n-1})$. Therefore, each element in $n$-mode Fock space is mapped to a unique $n$ spin state, and vise versa.

We can easily extend these ideas to 2D. For a square lattice on a torus each $\hat{Z}_i$ generator is mapped to $ZZZZ_i$ (a $Z$ on each edge adjacent to the $i$-th vertex), and the identity is mapped to itself, local gauge conditions around each plaquette $G_i$, and 2 non-local generators around the non-contractible loops ($X_V$, $X_H$):

$$\{I, (-1)^{d+1}X_V, (-1)^{d+1}X_H, G_0, \dots G_{p-1}, (-1)^{f_0}ZZZZ_0, \dots, (-1)^{f_n+d}ZZZZ_n\}$$

where there are $p$ plaquettes and $n$ fermionic modes (vertices). The $(-1)^{\sum_i^{n-1} f_i}ZZZZ_n$ generator can be constructed from a product of the first $n-1$ vertices so it is necessary to take $d = \sum_i^n f_i$ to stabilise a non-empty set. As there are the same number of plaquettes as vertices, there are $2n$ independent generators, and hence a unique $2n$ spin state is specified. This unique state is again parameterised by $(\sum_i^n f_i + 1, f_0, \dots, f_{n-1})$, and hence each element in $n$-mode Fock space is mapped to a unique $2n$ spin state.

Therefore, this is a unitary mapping between the $n$-mode Fock space and a gauge constrained subspace of $2n$ spins.

In the above discussion it is crucial that we were able to chose our boundary condition $d$ dependant on the state we were mapping, or else half of the Fock space would have been mapped to nothing. Therefore, it is essential to include this boundary condition as a degree of freedom on the spin side of the mapping in order to achieve unitarity.

# Chapter 3

# Quantum Metropolis Sampling

## 3.1 Introduction

State preparation is a major bottleneck for numerous quantum algorithms with applications in many fields including quantum chemistry [5, 78, 103, 55], quantum machine learning [1], quantum finance [20, 42, 52, 30], and quantum computational fluid dynamics [64]. The exponentially large state space afforded by a quantum computer that lends so much power when performing computations becomes a hindrance when we must initialise a state. In order to specify a **generic** $n$-qubit state **exactly** one must load $2^n$ amplitudes into our quantum computer, and hence any algorithms with a sub-exponential cost in $n$ become subdominant to the cost of state preparation.

Luckily slight relaxations of these requirements provide us with efficient avenues for state preparation. For example, considering the preparation of only a subset of possible states or by attempting to prepare states approximately through some form of lossy compression. Multiple techniques utilising these approaches have been developed including: preparing Matrix Product States (MPS) [91, 53, 81, 73, 34, 56, 74], sparse states [115], truncated series expansions [112, 118, 83], training variational circuits [117, 84, 75], compressing repeated values [104, 57], preparing states via coherent arithmetic [13, 47], and QSVT polynomial approximation [79, 43]. In this work we extend the QSVT polynomial approximation techniques to allow for piecewise QSVT polynomial approximations.

QSVT (Quantum Singular Value Transformation) [41] is a quantum algorithm that allows one to apply arbitrary polynomial transformations to block-encoded matrices. The flexibility of this algorithm allows it to approximate many other quantum processes, and hence has found numerous applications including amplitude amplification [41], Hamiltonian simulation [41], phase estimation [76], and linear system solvers [41].

Previous work utilised QSVT for state preparation to apply a global polynomial that approximates the amplitudes of a desired state, avoiding the need for coherent arithmetic and hence significantly reducing both the ancilla and Toffoli cost. Applications were demonstrated for a number of useful states including the Gaussian state and Kaiser window state [79]. However, this technique falls down when trying to simulate functions with sharp discontinuities or non-differentiable points. We navigate around these issues

by utilising piecewise polynomial approximations which simply deal with discontinuities by matching them to the boundaries of segments and cope with variability in approximation difficulty between regions by varying the length of segments used. As such, our techniques fair well where previous works failed, in particular we can efficiently load $\sqrt{x}$ which [79] explicitly noted was not possible with a single polynomial. Furthermore, we can load the B-spline window state with orders of magnitude lower cost than the Kaiser window state and we prove that this gives a similarly exponential performance boost to QPE.

In Section 3.2 we describe the technique for performing piecewise QSVT. First we present a new piecewise exact linear block encoding. Then, we demonstrate how to construct a QSVT circuit that implements piecewise polynomial transformations. We go on to discuss how to convert the transformed block encoding into a state and the resource costs of our quantum algorithm. In Section 3.3 we explore various applications of our algorithm to loading states of interest exponentially faster than a naive approach.

## 3.2 Piecewise QSVT

The aim of piecewise QSVT is to load a state with amplitudes described by different polynomials for different segments of computational basis elements:

$$\frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} p_{s_x} \left( 1 - 2 \frac{x \bmod L_{s_x}}{L_{s_x}} \right) |x\rangle \tag{3.1}$$

where $s_x \in \mathbb{Z}_S$ indexes which of the $S$ polynomial segments $x$ lies in, $L_{s_x}$ denotes the length of that segment, and $p_{s_x}$ is the polynomial transformation applied to that segment.

Piecewise QSVT uses a modified QSVT circuit (Section 3.2.2) to implement different polynomial transformations on different segments of a diagonal block encoded matrix. The diagonal (singular) values of the transformed block encoding are effectively samples from the polynomials taken for $x$-values corresponding to the original diagonal values. Hence, by knowing the values of the original diagonal block encoding: $1 - 2 \frac{x \bmod L_{s_x}}{L_{s_x}}$ (Section 3.2.1), and controlling the polynomials implemented: $p_{s_x}$ (Section 3.2.2), it is possible to produce a diagonal block encoding with values we desire: $p_{s_x} \left( 1 - 2 \frac{x \bmod L_{s_x}}{L_{s_x}} \right)$. By acting on a uniform superposition and performing amplitude amplification (Section 3.2.3), this diagonal block encoding can be easily converted into the desired state (3.1).

There are some conditions upon this process:

(1) $|p_i(t)| \leq 1$ for all $t \in [-1, 1]$ and $i \in \mathbb{Z}_S$

(2) $L_i$ must be a power of 2 ($L_i = 2^{l_i}$ for $l_i \in \mathbb{N}_0$)

(3) The $i$-th segment must span $x \in [aL_i, aL_i + (L_i - 1)]$ for some $a \in \mathbb{N}_0$

Condition (1) is inherited from QSVT, whereas conditions (2) and (3) are necessary for the pieces to be described by ignoring bits of $x$ which is key to allowing an efficient piecewise algorithm.

### 3.2.1 Piecewise exact linear block encoding

The singular values of our diagonal block encoding will become the $x$ values we wish to fit our polynomials at. As such it is desirable that our piecewise polynomials are each applied to singular values that span the whole range of possible values from 1 to $-1$. This makes it easier to ensure that the fitted polynomials satisfy condition (1) without rescaling. It also enables us to outperform the coherent inequality test based piecewise polynomial technique suggested in [79] which uses polynomials over disjoint domains between 0 and 1. In this case the polynomials may grow large outside the domains they approximate and hence could require large rescaling to satisfy condition (1) on the whole domain $[-1,1]$.

To achieve this we define a piecewise linear diagonal block encoding $U$ such that the $x$-th entry is given by $1-2\frac{x \bmod L_{s_x}}{L_{s_x}}$ where $L_{s_x}$ is the length of the segment containing $x$:

$$\left(\begin{array}{ccccccccc|c} 1 & & & & & & & & & \\ & 1-\frac{2}{L_0} & & & & & & & & \\ & & \ddots & & & & & & & \\ & & & -1+\frac{2}{L_0} & & & & & & \\ & & & & \ddots & & & & & \\ & & & & & 1 & & & & \\ & & & & & & 1-\frac{2}{L_{S-1}} & & & \\ & & & & & & & \ddots & & \\ & & & & & & & & -1+\frac{2}{L_{S-1}} & \\ \hline & & & & & & & & & \end{array}\right) \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \hline \end{array} \begin{array}{c} j_1 \\ \\ \\ \\ \\ \\ \\ \\ \\ \hline \end{array}$$

$$\begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \hline \end{array} \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \hline \end{array} \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \hline \end{array}$$

$$\begin{array}{ccccccccc|c} & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ \hline & & & & & & & & &

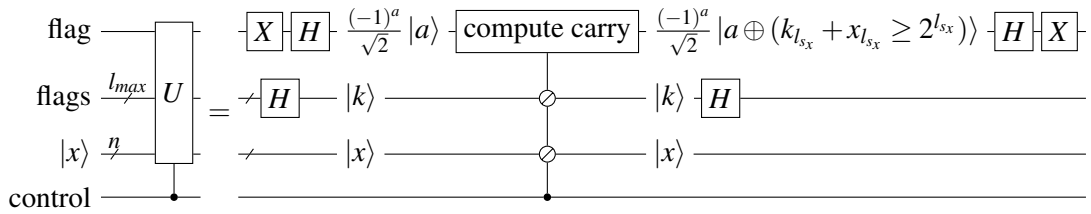



Fig. 3.1 Circuit implementing controlled version of piecewise exact linear block encoding where $L_{s_x} = 2^{l_{s_x}}$ and $l_{max} = \max_i(l_i)$. The slash circle control $\oslash$ is used to denote multiplexed control [72] (e.g. in this case it means that the action of the compute carry depends on the values of $k$ and $x$ in the corresponding registers). The mid circuit kets indicate the basis states per register with the full state being a summation over $a$ and $k$; after the application of compute carry these basis states are entangled.

It is simple to prove that Fig. 3.1 provides a block encoding of the desired matrix:

$$
\begin{aligned}
&(\langle 0| \otimes \langle 0|^{\otimes l_{max}} \otimes \langle x|)U(|0\rangle \otimes |0\rangle^{\otimes l_{max}} \otimes |x\rangle) \\
&= \sum_{a,b=0}^{1} \sum_{k,k'=0}^{L_{max}-1} \frac{(-1)^b \langle b| \langle k'|}{\sqrt{2L_{max}}} \\
&\qquad \frac{(-1)^a |a \oplus ((k_{l_{s_x}} + x_{l_{s_x}}) \geq 2^{l_{s_x}}))\rangle |k\rangle}{\sqrt{2L_{max}}} \\
&= \frac{1}{2L_{max}} \sum_{a=0}^{1} \sum_{k=0}^{L_{max}-1} (-1)^{a \oplus ((k_{l_{s_x}} + x_{l_{s_x}}) \geq 2^{l_{s_x}}))} (-1)^a \\
&= \frac{1}{L_{max}} \sum_{k=0}^{L_{max}-1} (-1)^{(k_{l_{s_x}} + x_{l_{s_x}}) \geq 2^{l_{s_x}}} \\
&= \frac{1}{2^{l_{s_x}}} \sum_{k=0}^{2^{l_{s_x}}-1} (-1)^{(k + x_{l_{s_x}}) \geq 2^{l_{s_x}}} \\
&= \sum_{k=0}^{2^{l_{s_x}} - x_{l_{s_x}} - 1} \frac{1}{2^{l_{s_x}}} - \sum_{k=2^{l_{s_x}} - x_{l_{s_x}}}^{2^{l_{s_x}}-1} \frac{1}{2^{l_{s_x}}} = \frac{2^{l_{s_x}} - 2x_{l_{s_x}}}{2^{l_{s_x}}} \\
&= 1 - 2\,\frac{x \bmod 2^{l_{s_x}}}{2^{l_{s_x}}}
\end{aligned}
$$

The adder costs $l_{max}$ compute/uncompute Toffoli pairs, the controlled variable unary iteration costs $S-1$ compute/uncompute Toffoli pairs, and the copy operations cost $S$ Toffolis. Therefore, the total cost for an $l_{max}$-compute carry, and hence the whole block encoding, is $l_{max} + 2S - 1$ Toffolis. As usual, we don't count uncomputation Toffolis, for which measurement based uncomputation can be used.

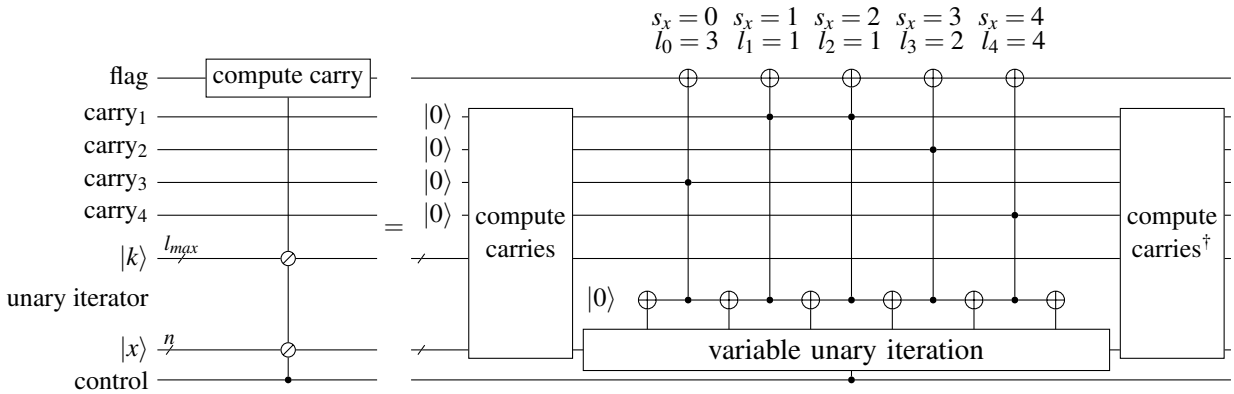



Fig. 3.2 Example of compute carry in Fig. 3.1 for $l_0 = 3$, $l_1 = 1$, $l_2 = 1$, $l_3 = 2$ and $l_4 = 4$. The unary iteration over these segments to copy the correct carry bit into the flag register is performed by controlled variable unary iteration [94]. Compared to Fig. 3.1, ancillas carry$_1$ to carry$_4$, and an ancilla unary iterator qubit are shown. The variable unary iteration contains further internal ancilla qubits.

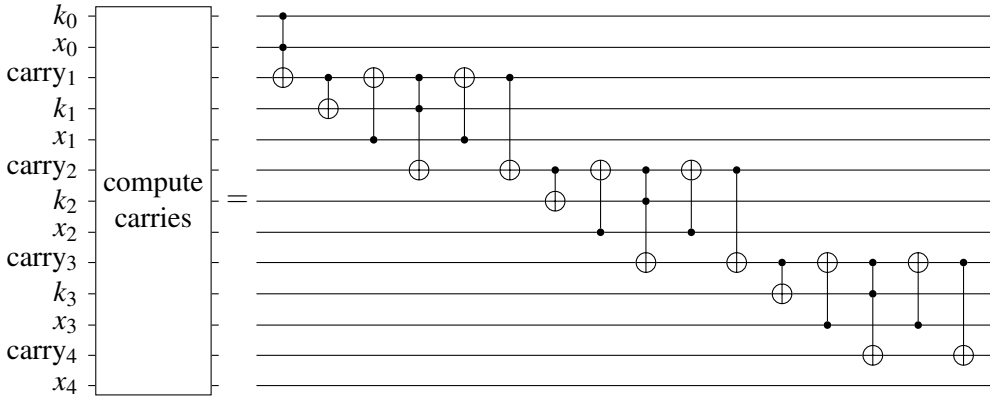



Fig. 3.3 Example of adder in Fig. 3.2 for $L_{max} = 16$, $N = 32$. It is important to note that the construction of the adder leaves the $x$ registers unaffected, so the variable unary iteration can operate directly on the $x$ register next. $x_4$ is not involved in this operation as $l_{max} = 4$ is less than $n = 5$ and it is not necessary to compute carries for $l > l_{max}$. The qubits have been ordered differently to Fig. 3.2 to make the ladder structure more apparent.

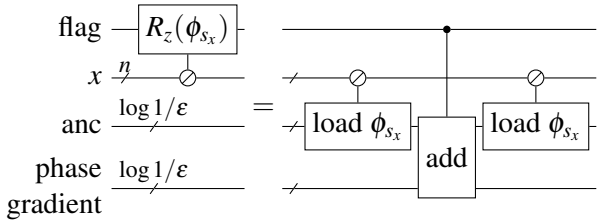



Fig. 3.4 Circuit demonstrating how the variable unary iteration controlled rotations can be implemented by digitally loading the phase factors and adding into a phase gradient register. This technique avoids Clifford+T compilation of rotations[14], which would lead to a multiplicative factor $\log 1/\varepsilon$ in T complexity.

### 3.2.2 Piecewise QSVT circuit

Standard QSVT applies a single polynomial to all the singular values of a block encoding. Here we wish to instead apply a different polynomial to each segment of our block encoding:

$$\left(\begin{array}{cccccc|c} p_0(1) & & & & & & \\ & \ddots & & & & & \\ & & p_0\left(-1+\frac{2}{L_0}\right) & & & & j_1 \\ & & & \ddots & & & \\ & & & & p_{S-1}(1) & & \\ & & & & & \ddots & \\ & & & & & p_{S-1}\left(-1+\frac{2}{L_{S-1}}\right) & \\ \hline j_2 & & & & & & j_3 \end{array}\right) \tag{3.2}$$

In QSVT the polynomial transformation performed is controlled by the choice of phase factor rotations. Hence by performing phase factor rotations conditionally depending on the values of the block registers we can apply different polynomial transformations to different segments of computational basis elements. As our block encoding is diagonal in the computational basis this allows us to apply different polynomials to different segments of our singular values.

Like the compute carry routine (Fig. 3.2 and Section 3.2.1), we utilise variable unary iteration [94] to efficiently apply different phase factor rotations for each section of computational basis elements. We achieve this by digitally pre-loading the phase factors into ancilla registers and then performing the rotations by controlled addition into a pre-prepared phase gradient register [40] as demonstrated by Fig. 3.4. Rather than inserting Fig. 3.4 into the QSVT circuit for each phase factor rotation, we can pre-load all of the phase factors in a single variable unary iteration, at the expense of more ancillas: Each set of controlled rotations requires their own set of $\log \frac{1}{\varepsilon}$ ancillas (where $\varepsilon$ is the accuracy to which one wishes to perform the rotations).

Utilising variable unary iteration to control off sections of computational basis elements makes our algorithm more efficient than applying different polynomials based on an inequality test as proposed in [79] which would cost $Sn$ Toffoli gates compared to our $S-2$ Toffoli gates.

We have based our circuit off an indefinite parity version of QSVT [105] which requires $2d$ queries to the block encoding and $2d+1$ rotations to implement a $d$-degree indefinite parity polynomial. We present

this variant as our later applications (Section 3.3) require indefinite parity. It would be equally possible to simply control the rotations of a standard QSVT circuit and restrict to definite parity polynomials [1]. Fig. 3.5 demonstrates a piecewise indefinite parity QSVT circuit with 4 iterations to implement $S$ different polynomials of degree 2 on $S$ different segments of length $2^{l_{s_x}}$.

Each step of the circuit requires $\log \frac{1}{\varepsilon} - 1$ compute/uncompute Toffoli pairs in order to perform the controlled rotation (via addition into a phase gradient register). The ancillas storing the phase factors must also be loaded/unloaded via variable unary iteration costing a total of $2(S-2)$ compute/uncompute Toffoli pairs. The remaining cost (not attributed to the inner block encoding) is due to the $l_{max}+1$-controlled Z gate in each step, costing a total of $2d(l_{max}+1)$ compute/uncompute Toffoli pairs.

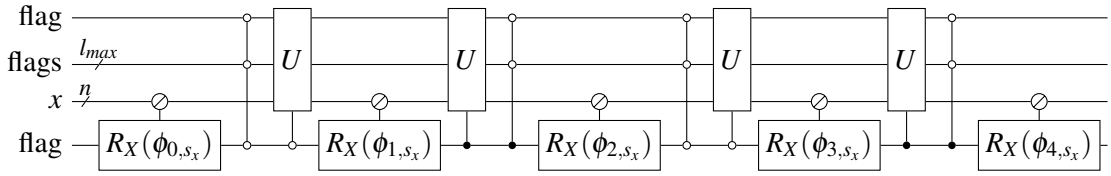



Fig. 3.5 Piecewise indefinite parity QSVT circuit with 4 iterations to implement $S$ different polynomials of degree 2 on $S$ different segments of length $2^{l_{s_x}}$. The multiplexed rotations are implemented efficiently using the variable unary iteration subroutine described in Section 3.2.1.

### 3.2.3 State preparation

Implementing the piecewise QSVT circuit produces the block encoding described by (3.2). The state produced by applying this block encoding (piecewise QSVT circuit) to the uniform superposition is:

$$\frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} p_{s_x} \left(1 - 2\frac{x \bmod 2^{l_{s_x}}}{2^{l_{s_x}}}\right) |x\rangle |0\rangle^{\otimes l_{max}+1} + |\text{junk}\rangle$$

Here $|\text{junk}\rangle$ corresponds to all the terms where the flag qubits are not in the all zero state. Therefore, we can use amplitude amplification (reflecting around the all zero state for the flag qubits) to recover the desired state (3.1). The number of amplitude amplification steps required to achieve this is given by:

$$O\left(\sqrt{\frac{N}{\sum_{x=0}^{N-1}\left[p_{s_x}\left(1-2\frac{x \bmod 2^{l_{sx}}}{2^{l_{sx}}}\right)\right]^2}}\right)$$

We always want to use polynomials that are as large as possible (under the restraint of condition (1)) to minimise the number of amplitude amplification steps, hence we fit polynomials $\tilde{p}_i$ to the normalised amplitudes of the desired state but actually implement polynomials $p_i = \frac{\tilde{p}_i}{\tilde{p}_{max}}$ (where $\tilde{p}_{max}$ is the maximum

[1] For definite parity polynomials it would be necessary to modify the piecewise exact linear block encoding to range from 0 to 1 rather than -1 to 1. This can be simply achieved by removing both Hadamard gates and the leftmost $X$ gate from the flag qubit in Fig. 3.1. Without this modification it would not be possible to approximate arbitrary states as the polynomial will be necessarily symmetric or anti-symmetric around 0.

value of all the polynomials over the continuous region $[-1,1]$). As $\sum_{x=0}^{N-1}\left[\tilde{p}_{s_x}\left(1-2\frac{x \bmod 2^{l_{sx}}}{2^{l_{sx}}}\right)\right]^2 \approx 1$ due to normalisation of the desired state, the number of amplitude amplification steps is given by: $O(\tilde{p}_{max}\sqrt{N})$.

For desired states with amplitudes given by samples from fixed functions, increasing $N$ corresponds to taking finer grained sampling of the function. As such the normalisation of such amplitudes will increase with $O(\sqrt{N})$. Hence if the maximum of the unnormalised polynomial approximation remains approximately the same with the finer sampling, the normalised maximum $\tilde{p}_{max}$ will scale with $O(\frac{1}{\sqrt{N}})$, and hence the number of amplitude amplification steps will scale with $O(1)$. This conclusion can similarly be reached by considering $\tilde{p}_{max}\sqrt{N}$ as the inverse "discretized L2-norm filling fraction" of $\tilde{p}_i$.

One caveat is that as $\tilde{p}_{max}$ is taken over the continuous region it may be larger than 1 even though all the normalised amplitudes are always less than 1. So for polynomials that oscillate wildly away from the sampled amplitudes this process is inefficient. It should be noted that fitting piecewise polynomials reduces the impact of the Runge Phenomena and hence polynomials produced by our approximations are less prone to this wild oscillation than in prior work.

### 3.2.4 Resource costs

The total cost of running a piecewise QSVT circuit to implement $S$ indefinite parity polynomials of degree $d$ with maximum segment size $2^{l_{max}}$ is $(2d+1)(\log\frac{1}{\varepsilon}-1)+2(S-2)+2d(l_{max}+1)+2d(l_{max}+2S-1)$ Toffolis.

When performing amplitude amplification with $A$ steps we repeat this circuit $2A$ times along with $2A$ single qubit rotations, $A$ times $(l_{max}+2)$-controlled Z gates and $A$ times $(n+l_{max}+2)$-controlled Z gates. Hence the cost of preparing a given $n$-qubit state is $O\left(\tilde{p}_{max}\sqrt{N}\max(n,dl_{max},d\log\frac{1}{\varepsilon},dS)\right)$ Toffolis. As discussed in Section 3.2.3, if the state we wish to load has amplitudes sampled from some function (such that increasing $N$ corresponds to taking a finer graining of the function), then the cost of preparing the $n$-qubit state is:

$$O\left(\max\left(n,dl_{max},d\log\frac{1}{\varepsilon},dS\right)\right) \text{ Toffolis}$$

In the case described in Section 3.3.1 we have $S=O(n)$, and hence this can be simplified to just:

$$O\left(d\max\left(n,\log\frac{1}{\varepsilon}\right)\right) \text{ Toffolis}$$

The algorithm acts on: $n$ qubits to store the desired state, $l_{max}+1$ block encoding flag qubits, $(2d+1)\log\frac{1}{\varepsilon}$ ancillas for digitally loading phase rotations, a QSVT flag qubit and an amplitude amplification ancilla qubit. Additional qubits are temporarily required: $n+l_{max}$ ancillas to perform the compute carry, $n$ ancillas to perform the variable unary iteration on $n$ qubits and $n+l_{max}+2$ ancillas to perform the $n+l_{max}+2$-controlled Z gate required in amplitude amplification. After accounting for the reuse of temporary ancillas, $2n+2l_{max}+(2d+1)\log\frac{1}{\varepsilon}+5$ total qubits are required by this algorithm.

Above we assumed the use of clean ancillas to minimise the Toffoli count; space-time tradeoffs using conditionally clean or dirty qubits are possible [62, 85].

## 3.3 Applications

Our algorithm allows more states to be prepared efficiently. Existing QSVT state preparation techniques required the desired state to be approximated by a single global polynomial. Our approach relaxes this restriction to allow approximation by piecewise polynomials. In Section 3.3.1 we will explore amplitudes sampled from functions which are hard to approximate with the former, but efficiently approximated by the later. In Section 3.3.2 we discuss loading the piecewise polynomial B-spline window function and the exponential performance boost it lends to Quantum Phase Estimation. Then in Section 3.3.3 we illustrate a classical algorithm for finding the optimal piecewise polynomial to approximately load a given generic state.

### 3.3.1 Approximating nearly-analytic functions

It is well known that analytic functions admit a $d$-degree polynomial approximation with approximation error $O(\exp(-d))$ [110]. Hence for such functions a single global polynomial will be sufficiently efficient. However, for functions with singularities on the interval we wish to approximate (e.g. $x^\alpha$ on $x, \alpha \in (0,1)$) it is not possible to find a single global polynomial that efficiently approximates the function. However, by utilising our technique with piecewise polynomials we can efficiently approximate some such functions with approximation error $O(\exp(-d))$ as demonstrated in Sections 3.3.1.1 and 3.3.1.2.

Our technique (similar to Section II E [94]) for dealing with such situations uses a cascade of exponentially smaller segments as we approach the singularity (e.g. $[2^{-i-1}, 2^{-i}]$ for a singularity at zero). This allows us to resolve the tricky behaviour up to a desired finite resolution $2^{-n}$ whilst only paying a logarithmic cost in number of segments $S = O(n)$. This technique will also be useful for many other states which exhibit large discontinuities, singularities or sharp cusps which would otherwise require a very high degree single polynomial to approximate.

In our examples we consider only the case with a single singularity at 0 when attempting to approximate the range $[2^{-n}, 1]$. This can be easily extended without loss of generality to any number of singularities within the interval of interest by dividing the interval up either side of each singularity at the closest multiples of $2^{-n}$. On an $n$-qubit state only a single amplitude can lie in each of the remaining $2^{-n}$ sized intervals, so they can be covered by single 0-degree polynomial segments.

We now illustrate 2 explicit examples of how our technique produces exponential speed ups in loading common functions compared to naive state prepartion and single polynomial QSVT. We also show an improvement over black box techniques for quantum state preparation [94, 11]. In general, the implementation of the black box oracle in each round of amplitude amplification can be costly: with quantum arithmetic evaluation of a piecewise polynomial this cost is $\Omega(n^2 d)$ [47], making our approach more efficient. Whilst we are unaware of specific quantum algorithms requiring these states, the examples effectively illustrate the increased capability of our approach compared to employing a single polynomial approximation.

#### 3.3.1.1 $x^\alpha$

In prior work [79] on QSVT state preparation it was explicitly stated that "one current drawback of our method is that it cannot efficiently prepare the state representing $\sqrt{x}$ for $x \in [0,1]$". Here we demonstrate that we can even approximately load the state $\frac{1}{\mathcal{N}}\sum_{x=0}^{2^n-1}(2^{-n}x)^\alpha |x\rangle$ for $\alpha \in (0,1)$ using our technique.

Theorem 8.1 in [110] states that any function $f$ analytic and bounded by $M$ on an open Bernstein ellipse $E_\rho$ with semi-major axis $\frac{\rho+\rho^{-1}}{2}$ has a $d$-degree Chebyshev approximation $f_d$ satisfying:

$$|f - f_d| \leq \frac{2M}{\rho - 1}\rho^{-d}$$

Considering the $S = n$ segments $[2^{-i-1}, 2^{-i}]$ separately for $f = x^\alpha$, then the maximal open Bernstein ellipse we can draw on each segment has $\rho = 3 + \sqrt{8}$. Therefore, we get the following bound on the error in each segment:

$$|f - f_{d,i}| \leq \frac{2 * 2^{-\alpha i}}{\mathcal{N}(2+\sqrt{8})}(3+\sqrt{8})^{-d}$$

Therefore, the $d$-degree $n$-piecewise approximation $f_d$ satisfies the following bound for $[2^{-n}, 1]$ as $\frac{1}{\mathcal{N}} < 1$ for $n > 1$.

$$|f - f_d| \leq \frac{2}{2+\sqrt{8}}(3+\sqrt{8})^{-d}$$

Further, we can utilise a constant approximation of 0 in the remaining segment $[0, 2^{-n})$.

The normalisation $\mathcal{N}$ is given by:

$$\sqrt{\sum_{x=0}^{2^n-1}(2^{-n}x)^{2\alpha}} = O\left(\sqrt{\frac{N}{2\alpha+1}}\right)$$

Hence, $\tilde{p}_{max} = O\left(\sqrt{\frac{2\alpha+1}{N}}\right)$. Therefore, our technique can efficiently prepare $x^\alpha$ for $\alpha \in (0,1)$ on $n$-qubits with accuracy $\varepsilon$ in $O(\sqrt{2\alpha}\log\frac{1}{\varepsilon}\max(n, \log\frac{1}{\varepsilon}))$ Toffolis — providing exponential speed-up on naive state preparation. We note that a faster technique [95] exists for preparing the special case $\sqrt{x}$, however, that approach does not generalise efficiently to $x^\alpha$.

#### 3.3.1.2 $\log x$

To approximately load the state $\frac{1}{\mathcal{N}}\sum_{x=1}^{2^n-1}\log 2^{-n}x\,|x\rangle$, we need to approximate the function $f(x) = \frac{1}{\mathcal{N}}\log 2^{-n}x$ for $x \in [1, 2^n - 1]$, or equivalently $\tilde{f}(x) = \frac{1}{\mathcal{N}}\log x$ for $x \in [2^{-n}, 1 - 2^{-n}]$.

Here, we will use a Taylor rather than Chebyshev approximation. The Taylor series expansion of $\tilde{f}$ around $x = a$ is given by:

$$\tilde{f}(x) = \sum_{k=0}^{d}\frac{\tilde{f}^{(k)}(a)}{k!}(x-a)^k + R_d(x,a)$$

where the Taylor remainder is

$$\begin{aligned} R_d(x,a) &= \frac{\tilde{f}^{(d+1)}(x_*)}{(d+1)!}(x-a)^{d+1} \\ &= \frac{1}{\mathscr{N}}\frac{(-1)^{d-1}}{d+1}x_*^{-d-1}(x-a)^{d+1} \end{aligned}$$

and $x_* \in (a,x)$.

We can split the range $[2^{-n+1},1]$ into segments $[2^{-i-1},3*2^{-i-2}]$ and $[3*2^{-i-2},2^{-i}]$ for $i \in \mathbb{Z}_{n-1}$, and then take a Taylor series expansion about $2^{-i-1}$ in the former segments and $2^{-i}$ in the latter. The maximal error $R_d$ occurs at $x_* = 2^{-i-1}$ and $x = 3*2^{-i-2}$ for the former segments, whereas it occurs at $x_* = 3*2^{-i-2}$ and $x = 3*2^{-i-2}$ for the latter segments. Therefore, the error in each segment is bounded by:

$$\begin{aligned} |R_{i,d}(x,a)| &\leq \frac{1}{\mathscr{N}(d+1)}2^{(i+1)(d+1)-(i+2)(d+1)} \\ &= \frac{1}{\mathscr{N}}\frac{2^{-d-1}}{d+1} \end{aligned}$$

Therefore, combining these into a single $2(n-1)$-piecewise Taylor series approximation $f_d$ achieves error bounded by:

$$|\tilde{f}-f_d| \leq \frac{1}{\mathscr{N}}\frac{2^{-d-1}}{d+1}$$

As $\frac{|\log 2^{-n}|}{\mathscr{N}} < 1$, we have

$$|\tilde{f}-f_d| \leq \frac{1}{n(d+1)}2^{-d-2}.$$

The above does not cover the range $[0,2^{-n+1})$, however, only two amplitudes lie in this range so they may be exactly approximated by a 1-degree polynomial, giving $S = 2n-1$ total segments.

The normalisation $\mathscr{N}$ is given by:

$$\sqrt{\sum_{x=0}^{2^n-1}\log^2 2^{-n}x} = O\left(\sqrt{N}\right)$$

Hence, $\tilde{p}_{max} = O\left(\frac{n}{\sqrt{N}}\right)$. Therefore, preparing $\log x$ on $n$-qubits with accuracy $\varepsilon$ costs $O(n\log\frac{1}{\varepsilon}\max(n,\log\frac{1}{\varepsilon}))$ Toffolis.

### 3.3.2 B-spline window

The B-spline window function [106] is a family of classical windowing functions constructed out of piecewise polynomials. The $m$-th B-spline $w_m(x)$ is obtained by $m$-fold self-convolution of the rectangular function:

$$w_m(x) = \int_{-\infty}^{\infty} w_1(mz)w_{m-1}\left(\frac{m}{m-1}(x-z)\right)dz \tag{3.3}$$

$$w_1(x) = \begin{cases} 1 & |x| < 2^{n-1} \\ 0 & |x| \geq 2^{n-1} \end{cases}$$

Hence the $m$-th B-spline is an $m$-piecewise polynomial of degree $m-1$. As such it is immediately obvious that our technique can prepare the following state (with accuracy $\varepsilon$) in $O(m^{\frac{5}{4}} \max(n, m, \log \frac{1}{\varepsilon}))$ Toffolis:

$$\frac{1}{\tilde{\mathcal{N}}} \sum_{x=2^{n-1}}^{2^{n-1}-1} w_m(x) |x\rangle \approx \frac{m^{m+1}}{2^{n(m-\frac{1}{2})}} \left(\frac{\pi}{3m}\right)^{\frac{1}{4}} \sum_{x=2^{n-1}}^{2^{n-1}-1} \sum_{p=0}^{m} \frac{(-1)^p \max\left(0, x - \left(p - \frac{m}{2}\right) \frac{2^n}{m}\right)^{m-1}}{p!\,(m-p)!} |x\rangle$$

Crucially, the $m$-fold self-convolution (3.3) corresponds to self-multiplying in the Fourier domain, and hence the Fourier transform of the $m$-th B-spline window function is:

$$\left(\frac{\pi}{3m}\right)^{\frac{1}{4}} \left(\frac{\sin \frac{x\pi}{m}}{\frac{x\pi}{m}}\right)^m$$

which has the useful property of being sharply peaked near $x = 0$.

Efficiently preparing the B-spline window function will be immensely useful for improving quantum algorithms. One of the main subroutines of quantum algorithms is Quantum Phase Estimation (QPE) which is used for finding the eigenvalues of a given unitary. The biggest problem with QPE is its fat tails; the probability of measuring an inaccurate eigenvalue is non-negligible if the eigenvalue is not guaranteed to be exactly specified in binary at the resolution used. This is typically countered through preparing a window function in the ancillas before applying QPE and increasing the resolution of the phase estimation beyond the precision desired. The window function has a narrow profile in the Fourier domain and hence narrows the profile of the probability distribution output by the QPE algorithm. One method for measuring the effectiveness of the window functions is by how much the resolution of the phase estimation must be increased by (or equivalently the number of extra ancillas required) to reduce the probability of measuring an energy outside the confidence interval to below a given value $\delta$ [44]. Extra ancilla qubits are included in the final prepared state, but unlike the base ancillas—which increase the resolution of the QPE—they serve instead to reduce the value of $\delta$. The current state of the art is the Kaiser window function [12] which reduces the number of extra ancillas required to $O(\log \log \frac{1}{\delta})$. Thereom 1 (proof in Appendix 3.6) demonstrates that the B-spline window achieves the same asymptotic complexity and hence provides a viable alternative to the Kaiser window:

**Theorem 1.** *To achieve a given probability $\delta$ of measuring energy outside of a given confidence interval $\varepsilon$ (when using $O(\log \frac{1}{\delta})$-th B-spline boosted QPE), one must use $O(\log \frac{1}{\varepsilon} + \log \log \frac{1}{\delta})$ ancillary qubits.*

Fig. 3.6 numerically demonstrates this doubly exponential decay. The exponent for the Kaiser window is larger than the B-Spline window, however, even with a very small number of extra ancillas both windows achieve such a small $\delta$ that it will likely be outweighed by other considerations such as logical error floor [3].

We compare the cost of preparing the B-spline window using our technique (with the improvements outlined in Section 3.4.1) to the cost of preparing the Kaiser window using the best technique known to us [79, 44]. Fig. 3.7 demonstrates that our technique provides a 50 fold decrease in preparation cost when using 4 extra ancillas. The code for producing both these figures can be found here: [88]. Therefore, in the early fault tolerant regime (where improvements in $\delta$ beyond experimental constraints cannot be realised) utilising the B-Spline window via our technique provides a significant reduction in cost for performing Quantum Phase Estimation.

It should be noted that in order to successfully prepare the B-spline window using our technique $m$ must be a power of 2, which happily corresponds to including an integer number of extra ancilla qubits.

The B-Spline function is an example of finding a piecewise polynomial that produces a desired property (being sharply peaked in the Fourier domain). Therefore, our technique offers a general framework by which one can efficiently prepare states with desired properties through careful choice of piecewise polynomials. This is potentially more powerful than simply approximating existing states known to have the desired property.

| Function | Toffoli cost |
|---|---|
| $\left(\frac{x}{N}\right)^{\alpha}, \alpha \in (0,1)$ | $O\left(\sqrt{2\alpha}\log\frac{1}{\varepsilon}\max(n,\log\frac{1}{\varepsilon})\right)$ |
| $\log\frac{x}{N}$ | $O\left(n\log\frac{1}{\varepsilon}\max(n,\log\frac{1}{\varepsilon})\right)$ |
| $m$-th B-spline window | $O\left(m^{\frac{5}{4}}\max\left(n,\log\frac{1}{\varepsilon}\right)\right)$ |

Table 3.1 Asymptotic Toffoli costs for loading these functions into the amplitudes of states on $n$ qubits with an error tolerance of $\varepsilon$. The tail probability $\delta$ when utilising the $m$-th B-spline window in QPE is given by $O(\exp(-m))$ as shown by Lemma 3.

### 3.3.3 Generic optimal algorithm

We present Algorithm 4 for finding the optimal splitting of the amplitudes into segments to be approximated that runs in $O(N\log_2 N)$ time. It takes a fixed degree $d$ and maximum error tolerance $\varepsilon$ and greedily finds the largest segment that a polynomial can approximate sequentially. This produces the smallest possible number of segments that satisfy the constraints, as for any given data point all the possible larger segments it lies in will have been considered and rejected by the algorithm.

Section 3.3.1 demonstrated upper bounds on the cost of approximating certain states by piecewise polynomials. In reality the chosen segment sizes are sub-optimal and we should instead run the algorithm presented here upon these states to find the optimal divisions into segments. The resulting approximation will improve upon the results demonstrated in Section 3.3.1.

[2]The tail probability depends only very weakly on the number of base qubits which correspond to how finely the window function is sampled. Hence, this figure is expected to be very similar for all choices of number of base qubits. This particular figure was generated for 10 base qubits.

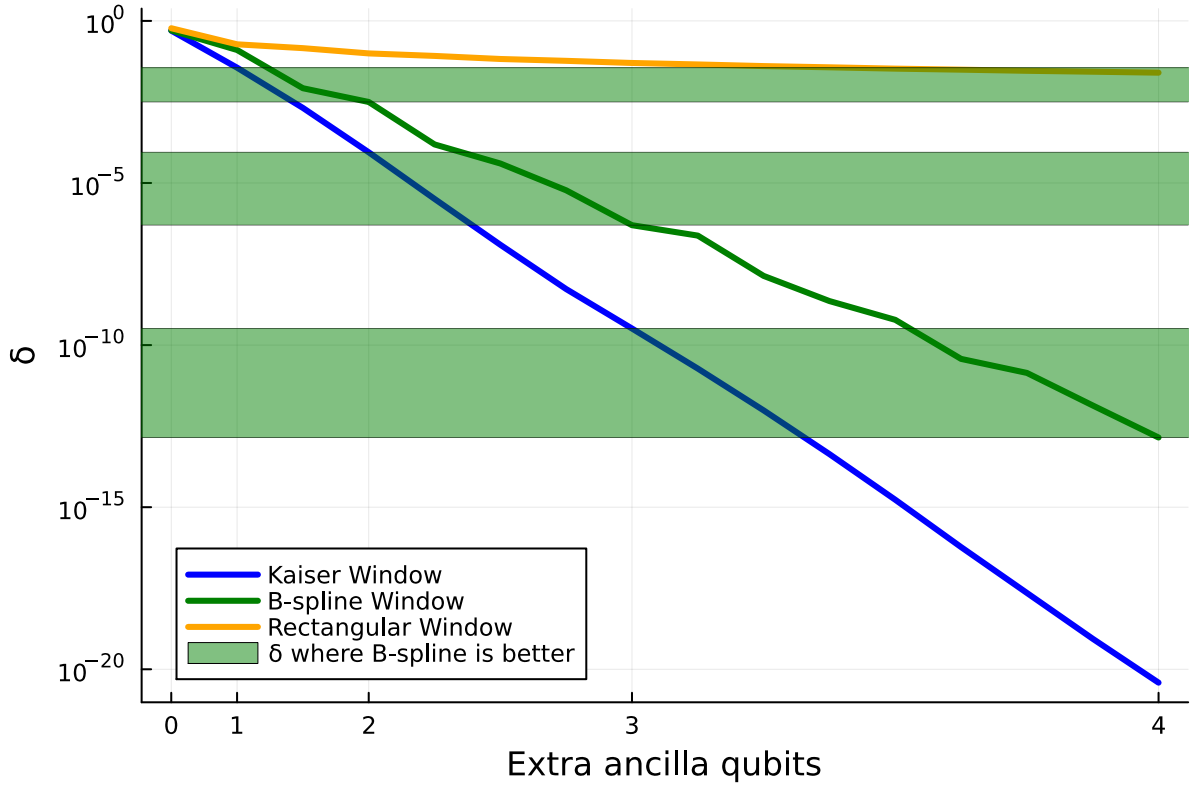



Fig. 3.6 Numerical results demonstrating $O(\log\log\frac{1}{\delta})$ dependence of extra ancilla qubits for QPE when using both the best Kaiser Window and the best B-Spline window [2]. Comparison with the rectangular window demonstrates the improvement over "unboosted" QPE. Both the Kaiser Window and B-Spline window demonstrate an exponential improvement, though the Kaiser window has a slightly larger exponent. The green regions indicate target error rates of $\delta$ for which the same number of extra ancillas are required for both the Kaiser window and the B-Spline window. In these regions it is optimal to use the B-Spline window due to its lower preparation cost. This figure includes calculations for fractional qubit counts as it is possible to calculate $\delta$ as if we used a confidence interval not equal to a power of 2.

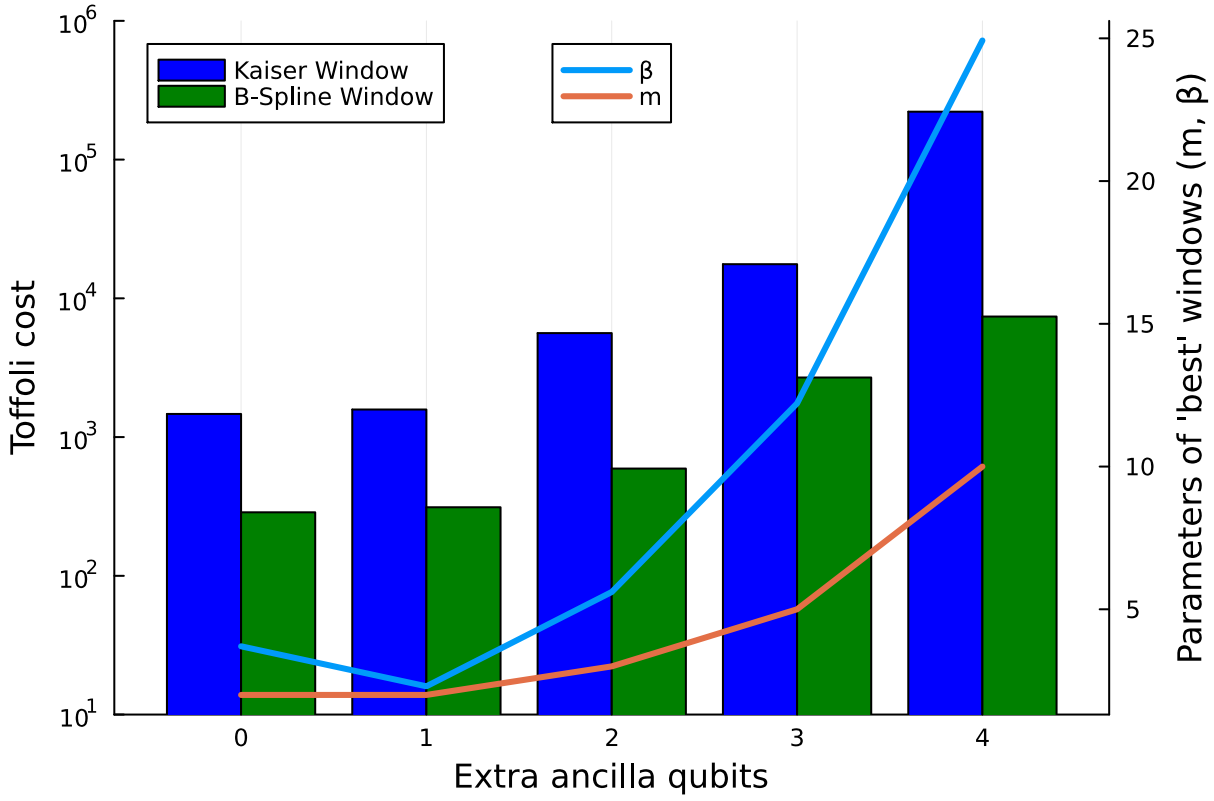



Fig. 3.7 Numerical results demonstrating Toffoli cost of preparing the “best" $\beta$-Kaiser and $m$-th B-Spline windows for a given number of extra ancilla qubits. The parameters ($\beta$ and $m$) of the windows are chosen to minimise the probability of measuring the wrong energy when performing QPE with 25 base qubits. Kaiser window is costed using the state-of-the-art approach from Appendix E of [44] (derived from [79]), whereas the B-spline window is costed using our piecewise QSVT according the formulas given in 3.2.4 incorporating the improvements from 3.4.1. These results demonstrate up to 50x lower cost for the B-spline window.

Our algorithm utilises an ERRPOLYAPPROX subroutine which finds a good polynomial approximation to the given set of amplitudes and returns the $L^\infty$ norm error of the approximation. We utilised least squares fitting to achieve this goal, but other approaches are feasible (e.g. polynomial frame approximations or regularised least squares). If it is known that the amplitudes are sampled from a particular function then other methods such as the Remez algorithm might be suitable.

**Algorithm 4** Performs at most $N \log_2 N$ queries to ERRPOLYAPPROX and returns the optimal cuts between the pieces of the polynomial

**Input:** $d$, $\varepsilon$ (target error), $N$, amplitudes
**Output:** cuts between polynomial segments

```
cuts ← [ ]
RHS ← N
while RHS > 0 do
    l ← max(l s.t. 2^l | RHS)
    LHS ← max(RHS − 2^l, 0)
    while ErrPolyApprox(amplitudes[LHS:RHS]) > ε do
        LHS ← (LHS+RHS)/2
    end while
    cuts.append(LHS)
    RHS ← LHS
end while
return cuts
```

## 3.4 Further improvements

### 3.4.1 Block Encoding

We can make our piecewise linear block encoding more efficient in some scenarios depending upon the sizes of segments we desire. If all the segments have the same size then no variable unary iteration is needed and a single Toffoli from the relevant carry to the flag qubit will suffice. This is also true if there is only one segment in which case our technique reproduces the prior work [79] with a new inner block encoding. In both these cases the cost of our inner block encoding is just $l_{max}+1$ Tofollis. This is significantly cheaper than inner block encodings introduced by previous work such as the sin [79] and approximate linear [43] blocking encodings as it avoids any compilation into Clifford+T of rotation gates[3]. Our block encoding does require more ancillas than the sin block encoding, but these ancillas will likely be required anyway by the subsequent algorithm so are well worth the $\approx 15$ times reduction in Toffolis.

Another efficiency saving can be made to the block encoding if there are only a small $k$ number of unique segment sizes. Here we could utilise variable unary iteration to load a label for each segment size into an ancilla register before performing QSVT and then control off this register (instead of the block

[3]To compare like for like we must slightly modify our block encoding to range from 0 to 1. This can be simply achieved by removing both Hadamard gates and the leftmost $X$ gate from the flag qubit in Fig. 3.1.

register) when copying the carries in compute carry. This would reduce the cost of the block encoding in each step of QSVT to $l_{max} + 2k - 1$.

### 3.4.2 Priors

We can reduce the number of amplitude amplification steps by applying our block encoding to an efficiently loaded prior distribution as in [65, 6]. By applying our QSVT circuit to the uniform superposition we are implicitly choosing a uniform prior. If we instead applied our circuit to the state $\sum_x c_x |x\rangle$, then we would use polynomial approximations to $a_x/c_x$ (where $a_x$ are the amplitudes of the state we wish to approximate). If $c_x$ is a good approximation then $a_x/c_x$ will be closer to the uniform distribution than $a_x$, so $\tilde{p}_{max}$ will be smaller and hence the number of amplitude amplification rounds will be reduced. This allows us to incorporate all the improvements demonstrated in [65] achieved by utilising efficiently preparable reference states as priors.

## 3.5 Conclusion

We have presented a new algorithm for preparing states that, in some explicit instances, demonstrates an exponential speedup over naive state preparation. In particular, our approach can handle nearly-analytic functions with cusps and singularities. Furthermore, we have demonstrated that the B-spline window function provides a viable alternative to the state-of-the-art Kaiser window for boosting QPE. We also present numerical results demonstrating that the B-spline function (implemented via our algorithm) is 50 times cheaper to prepare than the Kaiser window (implemented via existing state-of-the-art methods).

We expect that our algorithm in combination with the B-spline window will be adopted by early fault tolerant quantum phase estimation protocols when it will be crucial to reduce the Toffoli cost of circuits. QPE is a prolific component of numerous quantum algorithms and hence the impact of this improvement will be very widespread.

Further, our algorithm will prove useful for loading states with amplitudes drawn from smooth functions with sharp discontinuities. For example, loading a 2D or 3D grid of smooth data into the amplitudes of the 1D computational basis. This is a problem commonly encountered in Quantum Computational Fluid Dynamics [64].

In classical computer science, splines (piecewise polynomials) are an incredibly popular and powerful approximation technique and this is likely to be reflected in quantum computing. Hence, our algorithm which allows the preparation of spline states will likely find broad spread application.

We have only demonstrated our approach to loading states with real valued amplitudes. However, it is simple to extend our results to states with complex amplitudes. The only change we need to make is that now we approximate these states using piecewise polynomials with complex coefficients.

The downside of our approach is that our block encoding requires more flag and ancilla qubits than the prior work. We require $2n + 2l_{max} + 5$ qubits compared to $n + 4$ required by [79]. We argue that this extra qubit cost is acceptable for state preparation as it is highly likely that any subsequent algorithm will reuse these ancillas plus more so it will not increase the cost of the total quantum algorithm.

Additionally, it is relevant to note that in the regime of small quantum states (fewer than 9 base qubits), the method of Low, Kliuchnikov, and Schaeffer (LKS) for arbitrary state preparation [72] would be more efficient than our approach and that of McArdle et al. [79]. However, the cost of the LKS method increases rapidly with system size, making it less efficient than QSVT-based techniques even for moderately large states.

It is important to note that we do not apply QSVT piecewise to different segments of eigenvalues, but rather to different segments of Hilbert space enumerated by the computational basis elements. Here the later acts as the former because the eigenspaces of our block encoding align with the computational basis and the relationship between the eigenvalues and computational basis is known. In general this is not true and to perform different QSVT polynomial transformations to different sections of eigenvalues of an arbitrary block encoding would be much more costly and rely on some eigenvalue thresholding or windowing technique.

## 3.6 B-Spline Boosted Quantum Phase Estimation

Here we present a more detailed explanation of how the B-spline window function can be utilised to improve Quantum Phase Estimation.

Given an initial state $|\psi_i\rangle$ that is an eigenstate of $U$ with eigenvalue $E_i$. Standard QPE progresses by applying $U$ to the initial state conditional upon the value of $l$ ancilla qubits initialised in a uniform superposition. If instead the $l$ ancilla qubits are initialised with the B-spline window function then after this step we would have:

$$\frac{1}{\tilde{\mathcal{N}}} \sum_{x=-2^{l-1}+1}^{2^{l-1}} w_m(x) \, |x\rangle \, e^{iE_i x} \, |\psi_i\rangle$$

The next step of QPE is to apply the inverse Quantum Fourier Transform. The effect of this can be easily evaluated as multiplying by $e^{iE_i x}$ before a Fourier transform is equivalent to shifting by $E_i$ after a Fourier transform. As the Fourier transform of the $m$-th B-spline $w_m$ is given by [106], we have:

$$\frac{1}{\mathcal{N}} \sum_{k=-2^{l-1}+1}^{2^{l-1}} \left( \frac{\sin \frac{(k-E_i)\pi}{m}}{\frac{(k-E_i)\pi}{m}} \right)^m |k\rangle \, |\psi_i\rangle$$

As this distribution is sharply spiked near $k = E_i$, if we measure the ancilla register we are highly likely to get a value close to $E_i$.

We can prove this by bounding the tail probabilities, which first requires that we approximate $\mathcal{N}$.

**Lemma 2.** *Let $\mathcal{N}$ by defined as:*

$$\mathcal{N} := \sqrt{\sum_{k=-2^{l-1}+1}^{2^{l-1}} \left| \frac{\sin \frac{(k-E_i)\pi}{m}}{\frac{(k-E_i)\pi}{m}} \right|^{2m}}$$

*Then,*

$$\mathscr{N} \approx \left(\frac{3m}{\pi}\right)^{\frac{1}{4}}$$

*Proof.* We utilise the same technique as [12] used to approximate the normalisation of the Kaiser window, whereby we approximate the centre of the distribution by a Gaussian. This Gaussian is found by Taylor expanding the logarithm of the distribution:

$$\begin{aligned}\log\left(\frac{\sin\frac{(k-E_i)\pi}{m}}{\frac{(k-E_i)\pi}{m}}\right)^m &= m\log\frac{\sin\frac{(k-E_i)\pi}{m}}{\frac{(k-E_i)\pi}{m}} \\ &= -\frac{(k-E_i)^2\pi^2}{6m} + O\left((k-E_i)^4\right)\end{aligned}$$

Therefore, we can approximate the centre of the distribution by $e^{-\frac{x^2\pi^2}{6m}}$ and so we can utilise the normalisation for the Gaussian $\int_{-\infty}^{\infty}\left|e^{-\frac{x^2\pi^2}{6m}}\right|^2 dx = \sqrt{\frac{3m}{\pi}}$ to approximate the normalisation for our distribution. □

Now we must prove that the tail probability decreases exponentially as we increase $m$.

**Lemma 3.** *Let $\delta$ be the probability of measuring an energy $x$ outside of the confidence interval of $E_i \pm m$ when performing $m$-th B-spline boosted QPE. Then,*

$$\delta = O\left(\exp(-m)\right)$$

*Proof.* We define $\delta$ to be:

$$\delta := \sum_{|x-E_i|>m}\left|\frac{1}{\mathscr{N}}\left(\frac{\sin\frac{(x-E_i)\pi}{m}}{\frac{(x-E_i)\pi}{m}}\right)^m\right|^2$$

Therefore, we can bound $\delta$ as follows:

$$\begin{aligned}\delta &= \sum_{|x-E_i|>m}\left|\frac{1}{\mathscr{N}}\left(\frac{\sin\frac{(x-E_i)\pi}{m}}{\frac{(x-E_i)\pi}{m}}\right)^m\right|^2 \\ &\leq \sum_{|x-E_i|>m}\frac{1}{\mathscr{N}^2}\left(\frac{1}{\frac{(x-E_i)\pi}{m}}\right)^{2m} \\ &\leq \frac{2}{\mathscr{N}^2}\int_m^{\infty}\left(\frac{1}{\frac{x\pi}{m}}\right)^{2m} dx \\ &\leq \frac{2}{\mathscr{N}^2}\frac{m}{2m-1}\pi^{-2m}\end{aligned}$$

Therefore, using Lemma 2 we get:

$$\delta = O(\exp(-m))$$

□

Finally, we are ready to prove that the B-spline window function can boost QPE with the same asymptotic number of extra ancilla qubits as the Kaiser window function: $O\left(\log\log\frac{1}{\delta}\right)$.

**Theorem.** *To achieve a given probability $\delta$ of measuring energy outside of a given confidence interval $\varepsilon$ (when using $O(\log\frac{1}{\delta})$-th B-spline boosted QPE), one must use $O(\log\frac{1}{\varepsilon}+\log\log\frac{1}{\delta})$ ancillary qubits.*

*Proof.* Let $l$ be the number of ancilla qubits used by B-spline boosted QPE. Using Lemma 3, we can achieve a normalised confidence interval of $\frac{2m}{2^l}$ where $m = O(\log\frac{1}{\delta})$. Hence, $\varepsilon = \frac{O(\log\frac{1}{\delta})}{2^{l-1}}$. Therefore,

$$l = O\left(\log\left(\frac{1}{\varepsilon}\log\frac{1}{\delta}\right)\right)$$

□

# References


[1] Aaronson, S. (2015). Read the fine print. *Nature Phys.*, 11(4):291–293.

[2] Abrams, D. S. and Lloyd, S. (1997). Simulation of many-body fermi systems on a universal quantum computer. *Physical Review Letters*, 79(13):2586.

[3] Acharya, R. et al. (2024). Quantum error correction below the surface code threshold. *arXiv*.

[4] Aragão, M. P. d. and Werneck, R. F. (2002). On the implementation of mst-based heuristics for the steiner problem in graphs. In *Workshop on algorithm engineering and experimentation*, pages 1–15. Springer.

[5] Aspuru-Guzik, A., Dutoi, A. D., Love, P. J., and Head-Gordon, M. (2005). Simulated quantum computation of molecular energies. *Science*, 309(5741):1704–1707.

[6] Babbush, R., Berry, D. W., McClean, J. R., and Neven, H. (2019). Quantum simulation of chemistry with sublinear scaling in basis size. *npj Quantum Information*, 5(1).

[7] Backens, S., Shnirman, A., and Makhlin, Y. (2019). Jordan–wigner transformations for tree structures. *Scientific Reports*, 9(1).

[8] Ball, R. C. (2005). Fermions without fermion fields. *Phys. Rev. Lett.*, 95:176407.

[9] Banuls, M. C., Blatt, R., Catani, J., Celi, A., Cirac, J. I., Dalmonte, M., Fallani, L., Jansen, K., Lewenstein, M., Montangero, S., et al. (2020). Simulating lattice gauge theories within quantum technologies. *The European physical journal D*, 74(8):1–42.

[10] Barends, R., Lamata, L., Kelly, J., García-Álvarez, L., Fowler, A. G., Megrant, A., Jeffrey, E., White, T. C., Sank, D., Mutus, J. Y., et al. (2015). Digital quantum simulation of fermionic models with a superconducting circuit. *Nature communications*, 6(1):1–7.

[11] Bausch, J. (2022). Fast Black-Box Quantum State Preparation. *Quantum*, 6:773.

[12] Berry, D. W., Su, Y., Gyurik, C., King, R., Basso, J., Barba, A. D. T., Rajput, A., Wiebe, N., Dunjko, V., and Babbush, R. (2024). Analyzing Prospects for Quantum Advantage in Topological Data Analysis. *PRX Quantum*, 5(1):010319.

[13] Bhaskar, M. K., Hadfield, S., Papageorgiou, A., and Petras, I. (2016). Quantum algorithms and circuits for scientific computing. *Quant. Inf. Comput.*, 16(3-4):0197–0236.

[14] Bocharov, A., Roetteler, M., and Svore, K. M. (2015). Efficient synthesis of universal repeat-until-success quantum circuits. *Physical Review Letters*, 114(8).

[15] Bravyi, S. and Kitaev, A. (2002a). Fermionic quantum computation. *Annals of Physics*, 298(1):210–226. arXiv: quant-ph/0003137.

[16] Bravyi, S. B. and Kitaev, A. Y. (2002b). Fermionic quantum computation. *Annals of Physics*, 298(1):210–226.

[17] Bravyi, S. B. and Kitaev, A. Y. (2002c). Fermionic quantum computation. *Annals of Physics*, 298(1):210–226.

[18] Bultinck, N., Williamson, D. J., Haegeman, J., and Verstraete, F. (2017). Fermionic matrix product states and one-dimensional topological phases. *Phys. Rev. B*, 95:075108.

[19] Cao, Y., Romero, J., Olson, J. P., Degroote, M., Johnson, P. D., Kieferová, M., Kivlichan, I. D., Menke, T., Peropadre, B., Sawaya, N. P., et al. (2019). Quantum chemistry in the age of quantum computing. *Chemical reviews*, 119(19):10856–10915.

[20] Chakrabarti, S., Krishnakumar, R., Mazzola, G., Stamatopoulos, N., Woerner, S., and Zeng, W. J. (2021). A Threshold for Quantum Advantage in Derivative Pricing. *Quantum*, 5:463.

[21] Chakraborty, B., Honda, M., Izubuchi, T., Kikuchi, Y., and Tomiya, A. (2022). Classically emulated digital quantum simulation of the schwinger model with a topological term via adiabatic state preparation. *Physical Review D*, 105(9):094503.

[22] Chen, Y.-A. (2020). Exact bosonization in arbitrary dimensions. *Phys. Rev. Res.*, 2:033527.

[23] Chen, Y.-A. and Kapustin, A. (2019). Bosonization in three spatial dimensions and a 2-form gauge theory. *Phys. Rev. B*, 100:245127.

[24] Chen, Y.-A., Kapustin, A., and Radičević, Đ. (2018). Exact bosonization in two spatial dimensions and a new class of lattice gauge theories. *Annals of Physics*, 393:234–253.

[25] Chen, Y.-A. and Xu, Y. (2022). Equivalence between fermion-to-qubit mappings in two spatial dimensions. *arXiv preprint arXiv:2201.05153*.

[26] Chen, Y.-A. and Xu, Y. (2023). Equivalence between Fermion-to-Qubit Mappings in two Spatial Dimensions. *PRX Quantum*, 4(1):010326. Publisher: American Physical Society.

[27] Chiew, M. and Strelchuk, S. (2021). Optimal fermion-qubit mappings. *arXiv preprint arXiv:2110.12792*.

[28] Chiew, M. and Strelchuk, S. (2023). Discovering optimal fermion-qubit mappings through algorithmic enumeration. *Quantum*, 7:1145.

[29] Cirac, J. I., Pérez-García, D., Schuch, N., and Verstraete, F. (2021). Matrix product states and projected entangled pair states: Concepts, symmetries, theorems. *Rev. Mod. Phys.*, 93:045003.

[30] Dalzell, A. M. et al. (2023). End-To-End Resource Analysis for Quantum Interior-Point Methods and Portfolio Optimization. *PRX Quantum*, 4(4):040325.

[31] Derby, C., Klassen, J., Bausch, J., and Cubitt, T. (2021a). Compact fermion to qubit mappings. *Physical Review B*, 104(3):035118.

[32] Derby, C., Klassen, J., Bausch, J., and Cubitt, T. (2021b). Compact fermion to qubit mappings. *Phys. Rev. B*, 104:035118.

[33] Dijkstra, E. W. (2022). A note on two problems in connexion with graphs. In *Edsger Wybe Dijkstra: His Life, Work, and Legacy*, pages 287–290.

[34] Dilip, R., Liu, Y.-J., Smith, A., and Pollmann, F. (2022). Data compression for quantum machine learning. *Phys. Rev. Res.*, 4:043007.

[35] Fenwick, P. M. (1994). A new data structure for cumulative frequency tables. *Software: Practice and experience*, 24(3):327–336.

[36] Feynman, R. P. (2018). Simulating physics with computers. In *Feynman and computation*, pages 133–153. CRC Press.

[37] Fradkin, E. (1989). Jordan-wigner transformation for quantum-spin systems in two dimensions and fractional statistics. *Phys. Rev. Lett.*, 63:322–325.

[38] Friesner, R. A. (2005). Ab initio quantum chemistry: Methodology and applications. *Proceedings of the National Academy of Sciences*, 102(19):6648–6653.

[39] Gambetta, J. (2020). Ibms roadmap for scaling quantum technology.

[40] Gidney, C. (2018). Halving the cost of quantum addition. *Quantum*, 2:74.

[41] Gilyén, A., Su, Y., Low, G. H., and Wiebe, N. (2018). Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *51st Annual ACM SIGACT Symposium on Theory of Computing*.

[42] Gonzalez-Conde, J., Rodríguez-Rozas, A., Solano, E., and Sanz, M. (2023). Efficient Hamiltonian simulation for solving option price dynamics. *Phys. Rev. Res.*, 5(4):043220.

[43] Gonzalez-Conde, J., Watts, T. W., Rodriguez-Grasa, P., and Sanz, M. (2024). Efficient quantum amplitude encoding of polynomial functions. *Quantum*, 8:1297.

[44] Greenaway, S., Pol, W., and Sim, S. (2024). A case study against QSVT: assessment of quantum phase estimation improved by signal processing techniques. *arXiv*.

[45] Haegeman, J., Van Acoleyen, K., Schuch, N., Cirac, J. I., and Verstraete, F. (2015). Gauging quantum states: From global to local symmetries in many-body systems. *Phys. Rev. X*, 5:011024.

[46] Halperin, B. I., Lee, P. A., and Read, N. (1993). Theory of the half-filled landau level. *Phys. Rev. B*, 47:7312–7343.

[47] Häner, T., Roetteler, M., and Svore, K. M. (2018). Optimizing Quantum Circuits for Arithmetic. *arXiv*.

[48] Hastings, M. B., Wecker, D., Bauer, B., and Troyer, M. (2015). Improving quantum algorithms for quantum chemistry. *Quantum Info. Comput.*, 15(1–2):1–21.

[49] Havlíček, V., Troyer, M., and Whitfield, J. D. (2017). Operator locality in the quantum simulation of fermionic models. *Physical Review A*, 95(3):032332.

[50] Havlíček, V. c. v., Troyer, M., and Whitfield, J. D. (2017). Operator locality in the quantum simulation of fermionic models. *Phys. Rev. A*, 95:032332.

[51] Henderson, T., Chen, G. P., and Scuseria, G. E. (2022). Strong-weak duality via jordan-wigner transformation: Using fermionic methods for strongly correlated su (2) spin systems. *The Journal of Chemical Physics*.

[52] Herbert, S. (2021). No quantum speedup with grover-rudolph state preparation for quantum monte carlo integration. *Phys. Rev. E*, 103:063302.

[53] Holmes, A. and Matsuura, A. Y. (2020). Efficient quantum circuits for accurate state preparation of smooth, differentiable functions. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 169–179, Los Alamitos, CA, USA. IEEE Computer Society.

[54] Huerta, L. and Zanelli, J. (1993). Bose-fermi transformation in three-dimensional space. *Phys. Rev. Lett.*, 71:3622–3624.

[55] Huggins, W. J., Leimkuhler, O., Stetina, T. F., and Whaley, K. B. (2024). Efficient state preparation for the quantum simulation of molecules in first quantization. *arXiv*.

[56] Iaconis, J., Johri, S., and Zhu, E. Y. (2024). Quantum state preparation of normal distributions using matrix product states. *npj Quantum Inf.*, 10(1):15.

[57] Ivanov, A. V., Sünderhauf, C., Holzmann, N., Ellaby, T., Kerber, R. N., Jones, G., and Camps, J. (2023). Quantum computation for periodic solids in second quantization. *Phys. Rev. Res.*, 5:013200.

[58] Jain, J. K. (1989). Incompressible quantum hall states. *Phys. Rev. B*, 40:8079–8082.

[59] Jiang, Z., Kalev, A., Mruczkiewicz, W., and Neven, H. (2020). Optimal fermion-to-qubit mapping via ternary trees with applications to reduced quantum states learning. *Quantum*, 4:276.

[60] Jiang, Z., McClean, J., Babbush, R., and Neven, H. (2019). Majorana loop stabilizer codes for error mitigation in fermionic quantum simulations. *Phys. Rev. Appl.*, 12:064041.

[61] Jordan, P. and Wigner, E. P. (1993). Über das paulische äquivalenzverbot. In *The Collected Works of Eugene Paul Wigner*, pages 109–129. Springer.

[62] Khattar, T. and Gidney, C. (2024). Rise of conditionally clean ancillae for optimizing quantum circuits. *arXiv*.

[63] Kokail, C., Maier, C., van Bijnen, R., Brydges, T., Joshi, M. K., Jurcevic, P., Muschik, C. A., Silvi, P., Blatt, R., Roos, C. F., et al. (2019). Self-verifying variational quantum simulation of lattice models. *Nature*, 569(7756):355–360.

[64] Lapworth, L. (2022). A Hybrid Quantum-Classical CFD Methodology with Benchmark HHL Solutions. *arXiv*.

[65] Lemieux, J., Lostaglio, M., Pallister, S., Pol, W., Seetharam, K., Sim, S., and Şahinoğlu, B. (2024). Quantum sampling algorithms for quantum state preparation and matrix block-encoding. *arXiv*.

[66] Lloyd, S. (1996). Universal quantum simulators. *Science*, 273(5278):1073–1078.

[67] Lootens, L., Delcamp, C., Ortiz, G., and Verstraete, F. (2023a). Dualities in one-dimensional quantum lattice models: Symmetric hamiltonians and matrix product operator intertwiners. *PRX Quantum*, 4:020357.

[68] Lootens, L., Delcamp, C., and Verstraete, F. (2024a). Dualities in one-dimensional quantum lattice models: Topological sectors. *PRX Quantum*, 5:010338.

[69] Lootens, L., Delcamp, C., and Verstraete, F. (2024b). Entanglement and the density matrix renormalisation group in the generalised Landau paradigm. *arXiv*.

[70] Lootens, L., Delcamp, C., Williamson, D., and Verstraete, F. (2023b). Low-depth unitary quantum circuits for dualities in one-dimensional quantum lattice models.

[71] Lopez, A. and Fradkin, E. (1991). Fractional quantum hall effect and chern-simons gauge theories. *Phys. Rev. B*, 44:5246–5262.

[72] Low, G. H., Kliuchnikov, V., and Schaeffer, L. (2018). Trading T-gates for dirty qubits in state preparation and unitary synthesis.

[73] Lubasch, M., Joo, J., Moinier, P., Kiffner, M., and Jaksch, D. (2020). Variational quantum algorithms for nonlinear problems. *Phys. Rev. A*, 101:010301.

[74] Malz, D., Styliaris, G., Wei, Z.-Y., and Cirac, J. I. (2024). Preparation of matrix product states with log-depth quantum circuits. *Phys. Rev. Lett.*, 132:040404.

[75] Marin-Sanchez, G., Gonzalez-Conde, J., and Sanz, M. (2023). Quantum algorithms for approximate function loading. *Phys. Rev. Res.*, 5(3):033114.

[76] Martyn, J. M., Rossi, Z. M., Tan, A. K., and Chuang, I. L. (2021). Grand Unification of Quantum Algorithms. *PRX Quantum*, 2(4):040203.

[77] McArdle, S., Endo, S., Aspuru-Guzik, A., Benjamin, S. C., and Yuan, X. (2020a). Quantum computational chemistry. *Reviews of Modern Physics*, 92(1):015003.

[78] McArdle, S., Endo, S., Aspuru-Guzik, A., Benjamin, S. C., and Yuan, X. (2020b). Quantum computational chemistry. *Rev. Mod. Phys.*, 92:015003.

[79] McArdle, S., Gilyén, A., and Berta, M. (2022). Quantum state preparation without coherent arithmetic.

[80] McClean, J. R., Romero, J., Babbush, R., and Aspuru-Guzik, A. (2016). The theory of variational hybrid quantum-classical algorithms. *New Journal of Physics*, 18(2):023023.

[81] Melnikov, A. A., Termanova, A. A., Dolgov, S. V., Neukart, F., and Perelshtein, M. R. (2023). Quantum state preparation using tensor networks. *Quantum Science and Technology*, 8(3):035027.

[82] Miller, A., Zimborás, Z., Knecht, S., Maniscalco, S., and García-Pérez, G. (2023). Bonsai algorithm: Grow your own fermion-to-qubit mappings. *PRX Quantum*, 4:030314.

[83] Moosa, M., Watts, T. W., Chen, Y., Sarma, A., and McMahon, P. L. (2024). Linear-depth quantum circuits for loading Fourier approximations of arbitrary functions. *Quantum Sci. Technol.*, 9(1):015002.

[84] Nakaji, K., Uno, S., Suzuki, Y., Raymond, R., Onodera, T., Tanaka, T., Tezuka, H., Mitsuda, N., and Yamamoto, N. (2022). Approximate amplitude encoding in shallow parameterized quantum circuits and its application to financial market indicators. *Phys. Rev. Res.*, 4(2):023136.

[85] Nie, J., Zi, W., and Sun, X. (2024). Quantum circuit for multi-qubit Toffoli gate with optimal resource. *arXiv*.

[86] Nishimori, H. and Ortiz, G. (2010). *Elements of phase transitions and critical phenomena*. Oup Oxford.

[87] O'Brien, O. and Strelchuk, S. (2024). Ultrafast hybrid fermion-to-qubit mapping. *Phys. Rev. B*, 109:115149.

[88] O'Brien, O. and Sünderhauf, C. (2025). Quantum state preparation via piecewise qsvt code. Zenodo.

[89] Ortiz, G., Gubernatis, J., Knill, E., and Laflamme, R. (2002). Simulating fermions on a quantum computer. *Computer physics communications*, 146(3):302–316.

[90] Ortiz, G., Gubernatis, J. E., Knill, E., and Laflamme, R. (2001). Quantum algorithms for fermionic simulations. *Physical Review A*, 64(2):022319.

[91] Ran, S.-J. (2020). Encoding of matrix product states into quantum circuits of one- and two-qubit gates. *Physical Review A*, 101(3).

[92] Rehfeldt, D. and Koch, T. (2021). Implications, conflicts, and reductions for steiner trees. *Mathematical Programming*, pages 1–64.

[93] Reiher, M., Wiebe, N., Svore, K. M., Wecker, D., and Troyer, M. (2017). Elucidating reaction mechanisms on quantum computers. *Proceedings of the national academy of sciences*, 114(29):7555–7560.

[94] Sanders, Y. R., Berry, D. W., Costa, P. C., Tessler, L. W., Wiebe, N., Gidney, C., Neven, H., and Babbush, R. (2020). Compilation of fault-tolerant quantum heuristics for combinatorial optimization. *PRX Quantum*, 1(2).

[95] Sanders, Y. R., Low, G. H., Scherer, A., and Berry, D. W. (2019). Black-box quantum state preparation without arithmetic. *Phys. Rev. Lett.*, 122:020502.

[96] Seeley, J. T., Richard, M. J., and Love, P. J. (2012a). The bravyi-kitaev transformation for quantum computation of electronic structure. *The Journal of chemical physics*, 137(22):224109.

[97] Seeley, J. T., Richard, M. J., and Love, P. J. (2012b). The Bravyi-Kitaev transformation for quantum computation of electronic structure. *The Journal of Chemical Physics*, 137(22):224109. arXiv: 1208.5986.

[98] Setia, K., Bravyi, S., Mezzacapo, A., and Whitfield, J. D. (2019a). Superfast encodings for fermionic quantum simulation. *Physical Review Research*, 1(3):033033.

[99] Setia, K., Bravyi, S., Mezzacapo, A., and Whitfield, J. D. (2019b). Superfast encodings for fermionic quantum simulation. *Physical Review Research*, 1(3).

[100] Shukla, S. K., Ellison, T. D., and Fidkowski, L. (2020). Tensor network approach to two-dimensional bosonization. *Phys. Rev. B*, 101:155105.

[101] Steudtner, M. and Wehner, S. (2019a). Quantum codes for quantum simulation of fermions on a square lattice of qubits. *Physical Review A*, 99(2):022308.

[102] Steudtner, M. and Wehner, S. (2019b). Quantum codes for quantum simulation of Fermions on a square lattice of qubits. *Physical Review A*, 99(2):022308. arXiv:1810.02681 [quant-ph].

[103] Su, Y., Berry, D. W., Wiebe, N., Rubin, N., and Babbush, R. (2021). Fault-tolerant quantum simulations of chemistry in first quantization. *PRX Quantum*, 2:040332.

[104] Sünderhauf, C., Campbell, E., and Camps, J. (2024). Block-encoding structured matrices for data input in quantum computing. *Quantum*, 8:1226.

[105] Sünderhauf, C. (2023). Generalized quantum singular value transformation. Upcoming version.

[106] Toraichi, K., Kamada, M., Itahashi, S., and Mori, R. (1989). Window functions represented by b-spline functions. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 37(1):145–147.

[107] Tranter, A., Love, P. J., Mintert, F., and Coveney, P. V. (2018). A comparison of the bravyi–kitaev and jordan–wigner transformations for the quantum simulation of quantum chemistry. *Journal of chemical theory and computation*, 14(11):5617–5630.

[108] Tranter, A., Love, P. J., Mintert, F., Wiebe, N., and Coveney, P. V. (2019). Ordering of trotterization: Impact on errors in quantum simulation of electronic structure. *Entropy*, 21(12):1218.

[109] Tranter, A., Sofia, S., Seeley, J., Kaicher, M., McClean, J., Babbush, R., Coveney, P. V., Mintert, F., Wilhelm, F., and Love, P. J. (2015). The Bravyi–Kitaev transformation: Properties and applications. *Int. J. Quant. Chem.*, 115(19):1431–1441.

[110] Trefethen, L. N. (2019). *Approximation Theory and Approximation Practice, Extended Edition*. Society for Industrial and Applied Mathematics, Philadelphia, PA.

[111] Verstraete, F. and Cirac, J. I. (2005). Mapping local hamiltonians of fermions to local hamiltonians of spins. *Journal of Statistical Mechanics: Theory and Experiment*, 2005(09):P09012.

[112] Welch, J., Greenbaum, D., Mostame, S., and Aspuru-Guzik, A. (2014). Efficient quantum circuits for diagonal unitaries without ancillas. *New Journal of Physics*, 16(3):033040.

[113] Whitfield, J. D., Havlíček, V. c. v., and Troyer, M. (2016). Local spin operators for fermion simulations. *Phys. Rev. A*, 94:030301(R).

[114] Xu, H., Chung, C.-M., Qin, M., Schollwöck, U., White, S. R., and Zhang, S. (2023). Coexistence of superconductivity with partially filled stripes in the hubbard model.

[115] Zhang, X.-M., Li, T., and Yuan, X. (2022). Quantum state preparation with optimal circuit depth: Implementations and applications. *Phys. Rev. Lett.*, 129:230504.

[116] Zohar, E. and Cirac, J. I. (2018). Eliminating fermionic matter fields in lattice gauge theories. *Phys. Rev. B*, 98:075119.

[117] Zoufal, C., Lucchi, A., and Woerner, S. (2019). Quantum generative adversarial networks for learning and loading random distributions. *npj Quantum Information*, 5(1).

[118] Zylberman, J. and Debbasch, F. (2024). Efficient quantum state preparation with Walsh series. *Phys. Rev. A*, 109(4):042401.

# Appendix A

# Fermion-to-Qubit Mappings

## A.1 Hybrid Mappings

### A.1.1 Steiner Tree Solver

In our analysis we evaluate the performance of different mappings on a square-lattice qubit architecture. Some mappings require 2-qubit gates to be performed on non-neighbouring qubits and so we need additional SWAP gates. Calculating the optimum choice of SWAP gates requires finding the minimum path length required to connect all the qubits involved in a given fermionic operation. This is an example of an NP-hard problem known as the Steiner Tree problem:

---

STEINER TREE PROBLEM

GIVEN AN UNDIRECTED CONNECTED GRAPH $G = (V, E)$ AND A SET $T \in V$ OF TERMINALS, FIND A TREE $S \in G$ OF MINIMUM LENGTH THAT SPANS $T$

---

In our case the set of terminals corresponds to the set of qubits involved in the given fermionic operation which we want to find the minimum path of SWAP gates between. In our calculations we use a top-ranking Steiner Tree solver SCIP-JACK [92] to find this minimum path. SCIP-JACK consists of three component parts: preprocessing, primal heuristics, and a branch-and-cut procedure.

The preprocessing uses reduction techniques to reduce the size of the graph whilst still allowing at least one solution to the reduced problem to be converted into an optimal solution to the original graph. Examples of these techniques include merging together connected vertices that share no other neighbours or discarding 1-degree non-terminal vertices.

The primal heuristic finds good or optimal solutions to the problem. In SCIP-JACK the one-phase repetitive shortest path heuristic [4] is used. This starts from a vertex and uses Dijkstra's algorithm [33] to find the shortest path to the nearest terminal. This procedure is then reiterated with the modification to Dijkstra's algorithm that the tentative distance assigned to each vertex is carried over. Once this has built up a subgraph containing all the terminals, a minimum spanning tree is constructed covering the subgraph, and any 1-degree non-terminal vertices are pruned.

The branch-and-cut procedure is the backbone of the program used to compute a lower bound and prove the optimality of the solution. First, the problem is expressed in terms of a integer programming problem by considering $x_e$ to be binary variables representing whether edge $e \in E$ is included in the Steiner tree:

STEINER TREE PROBLEM UNDIRECTED CUT FORMULATION

Define $\delta(U) = \{\{u,v\} \in E : u \in U, v \in V/U\}$

Find min $\sum_e x_e$ under the following constraints:

1: $x(\delta(W)) \geq 1$ for all $W \in V$, $0 < |W \cap T| < |T|$

2: $x_e \in \{0,1\} \forall e \in E$

Linear programming is used to initially optimise the solution when considering only a subset of these constraints providing a lower bound on an optimal solution. The primary heuristic is then run with edge costs given by $1 - x_e$ to find a feasible solution (reasonably similar to the LP solution) providing an upper bound on the optimal solution. Then, the algorithm considers the restraints ("cuts") that the current solution violates and adds them to the subset of constraints in the linear programming model. This model is repeatedly solved and expanded until the subset of constraints has grown sufficiently. At this point, if the solution is integral then we have found an optimal solution, otherwise the algorithm "branches" by creating two new sub-problems (nodes) each with an additional constraint on one of the variables ($x_e$) that forces it to be either 0 or 1 in each problem respectively. The process is then repeated again on each of these nodes (potentially with more constraints added). If for any node the optimal solution exceeds any of the heuristic solutions found, the node is discarded (pruned).

### A.1.2 Visual guide to Hybrid mappings

In this section we provide a series of diagrams to further elaborate and explain our hybrid mapping algorithms.

Figures A.1.1 and A.1.2 respectively describe the Jordan-Wigner mapping over the roots of Bravyi-Kitaev cells in the Hybrid and Hybrid+ mapping schemes.

Figure A.1.3 illustrates the family of stabilisers utilised by the Hybrid+ mapping scheme. This is identical to the stabilisers found in [101].

Figures A.1.4 and A.1.5 demonstrate how the Hybrid+ mapping scheme shortens different Pauli-strings.

In the figures below: the dashed boxes represent the cells, whilst the circles inside them represent the roots of each cell, and the circles without boxes represent the ancillas.

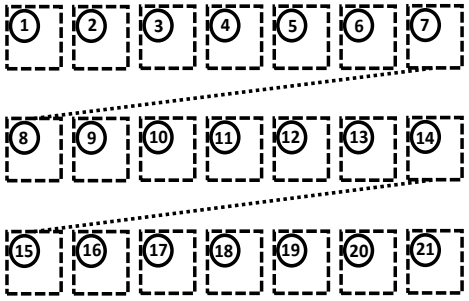



Fig. A.1.1 Diagram illustrating how the *cell number* is assigned in a Z pattern to each cell in the Hybrid mapping.

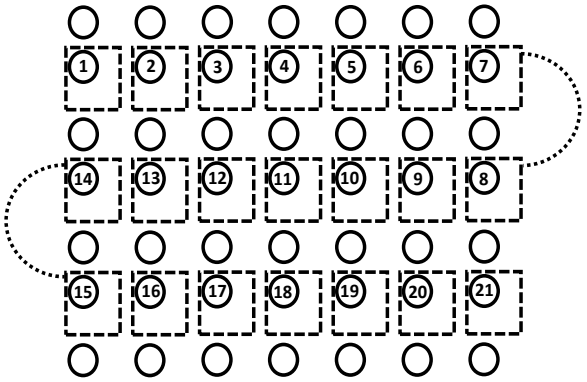



Fig. A.1.2 Diagram illustrating how the *cell number* is assigned in a S pattern to each cell in the Hybrid+ mapping. The unnumbered circles represent the ancillas each connected to the cell directly below them.

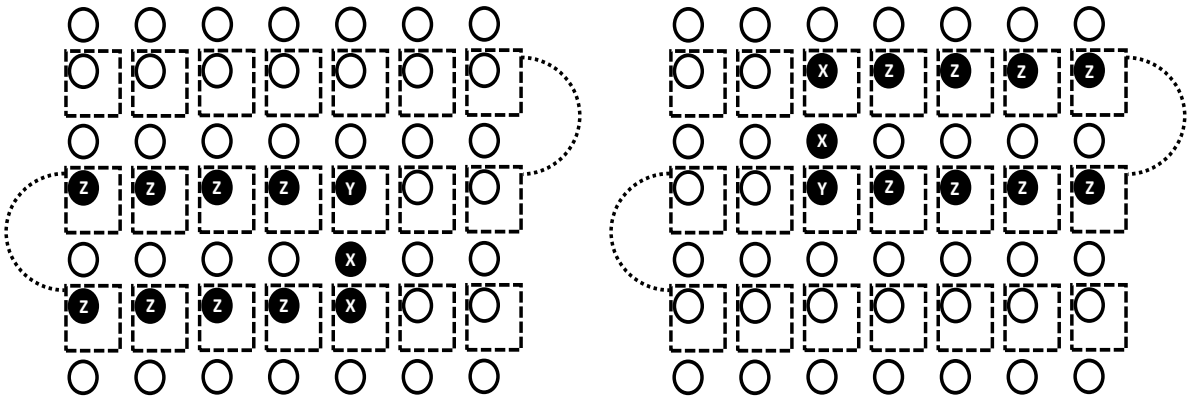



Fig. A.1.3 Examples from the family of stabilisers created by applying the series of gates illustrated in 2.6 and 2.7. The family consists of a stabiliser for every ancilla which is a string of Z gates that follow the line of the S-pattern with either $XXY$ or $YXX$ gates surrounding the ancilla depending on whether the row is odd or even.

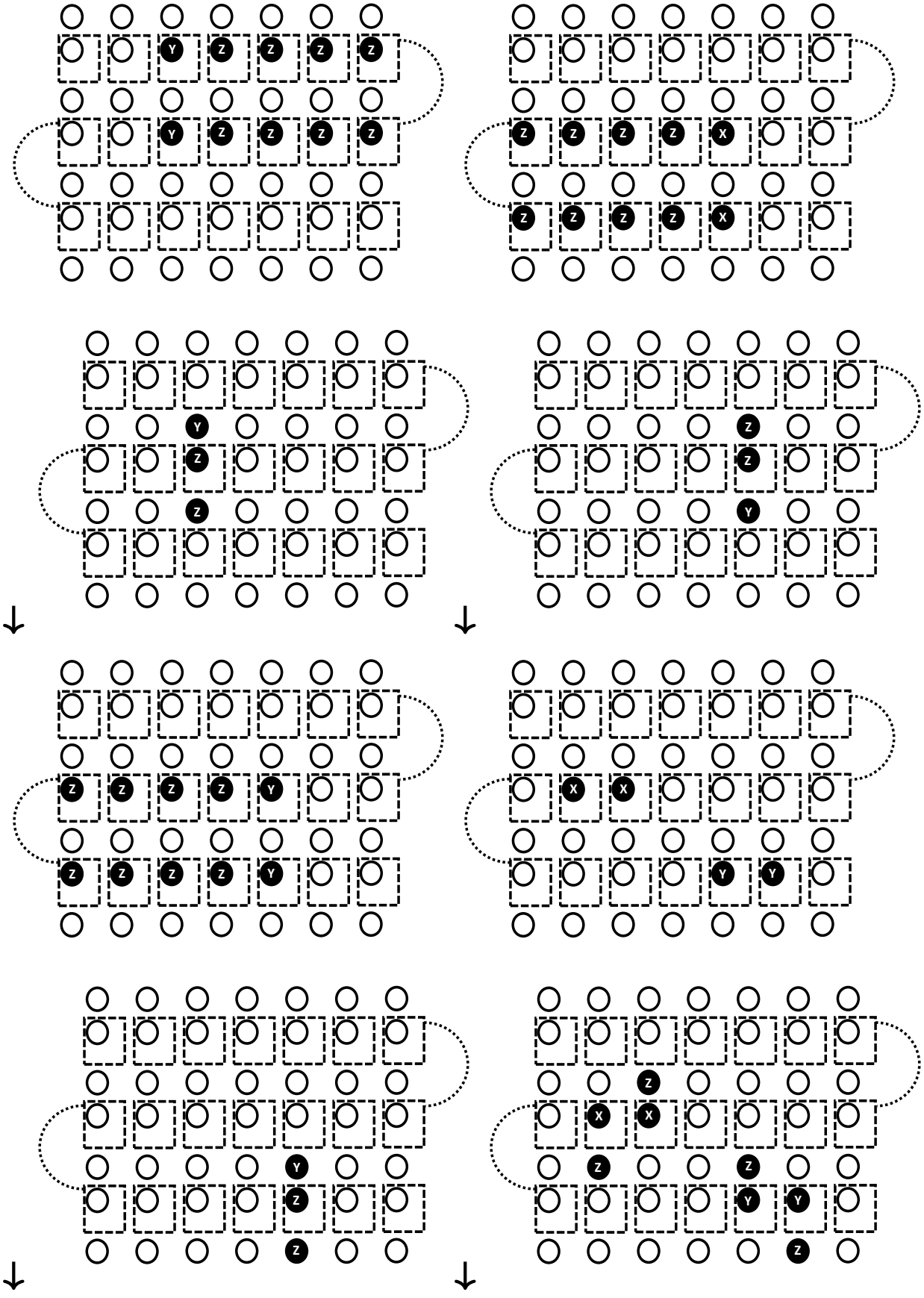


Fig. A.1.4 Diagrams showing how the portion of Pauli-strings acting on the roots of cells are modified and simplified by the Hybrid+ mapping algorithm

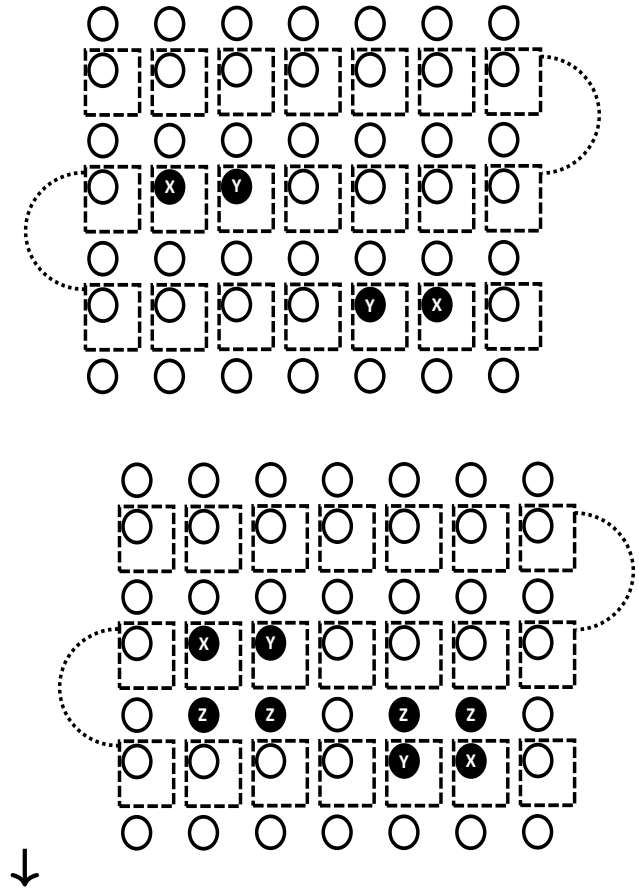



Fig. A.1.5 Diagrams showing how the portion of a Pauli-string acting on the roots of the cells is modified and simplified by the Hybrid+ mapping algorithm

# Appendix B

# Installing the CUED class file

LaTeX.cls files can be accessed system-wide when they are placed in the <texmf>/tex/latex directory, where <texmf> is the root directory of the user's TeXinstallation. On systems that have a local texmf tree (<texmflocal>), which may be named "texmf-local" or "localtexmf", it may be advisable to install packages in <texmflocal>, rather than <texmf> as the contents of the former, unlike that of the latter, are preserved after the LaTeXsystem is reinstalled and/or upgraded.

It is recommended that the user create a subdirectory <texmf>/tex/latex/CUED for all CUED related LaTeXclass and package files. On some LaTeXsystems, the directory look-up tables will need to be refreshed after making additions or deletions to the system files. For TeXLive systems this is accomplished via executing "texhash" as root. MIKTeXusers can run "initexmf -u" to accomplish the same thing.

Users not willing or able to install the files system-wide can install them in their personal directories, but will then have to provide the path (full or relative) in addition to the filename when referring to them in LaTeX.